

数据库系统课程设计报告

小组名称：待填写真实小组名称

小组成员：待提供真实名单

撰写时间：2026 年 11 月 13 日

文档版本：V1.0

目录

一、成员分工.....3

二、需求分析.....4

三、系统概念结构设计.....11

四、系统的逻辑结构设计.....25

五、系统物理结构设计、持久层设计和关键业务代码.....31

六、项目运维管理和优化.....63

七、附件内容.....83

一、成员分工

本项目的网站名称为“左邻右舍”，使用 Java 和 MySQL 实现社区居家养老服务管理，按档案评估、服务派单、护工排班、健康监测和补贴监管五个模块组织。

【待确认】下面为分工建议。请填写组长、成员姓名学号，并按实际完成的工作调整。全文蓝色文字为待确认或待补充内容。

业务模块	建议负责内容	负责人
档案与评估	用例与鲁棒图、局部数据模型、SQL、视图、过程、触发器和测试	待填写
下单与派单	用例与鲁棒图、局部数据模型、SQL、视图、过程、触发器和测试	待填写
护工与排班	用例与鲁棒图、局部数据模型、SQL、视图、过程、触发器和测试	待填写
监测与应急	用例与鲁棒图、局部数据模型、SQL、视图、过程、触发器和测试	待填写
补贴与监管	用例与鲁棒图、局部数据模型、SQL、视图、过程、触发器和测试	待填写

【待补充】小组共 4—5 人，成员名单、各人完成情况和协作记录待填写。

二、需求分析

老人完成建档和能力评估后，社区为其制定照护计划。家属依据计划预约服务，管理员安排护工上门。服务结束后，系统保存服务记录、工时和家属确认结果，作为后续补贴核算的依据。设备监测和应急处置也关联到老人档案。设计中需要重点处理评估过期、重复派单、服务时间冲突、设备换绑和补贴重复申报等问题。

参与者包括社区管理员、老人及家属、护工、服务机构负责人、街道与民政监管。街道初审和民政终审在账号角色上细分。家属只访问关联老人，护工只访问本人订单，机构负责人只管理本机构护工，监管人员不能任意修改原始服务事实。

档案与评估

管理员建立唯一档案和家属关联，登记有效评估，再选择匹配能力等级的照护包并制定计划。每份计划引用采用的评估，系统检查有效期覆盖与区间重叠。调整计划建立新版本并保留旧频次。登记当前复评后暂停旧计划，后续服务采用最新已生效评估；复评通知只针对最新有效评估生成；家属可查看授权联系方式和计划项目。

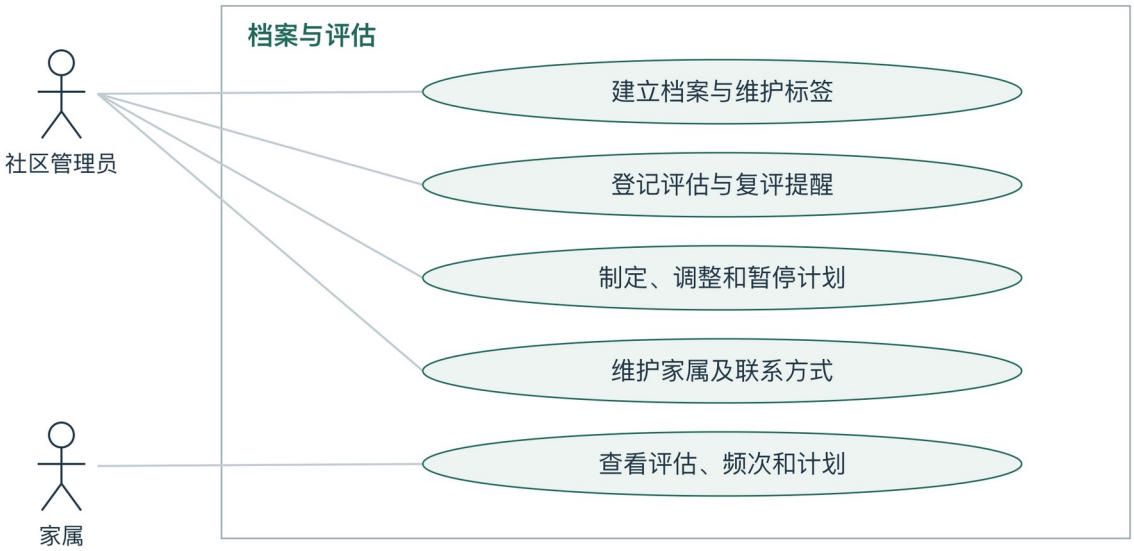


图 1 档案与评估用例图

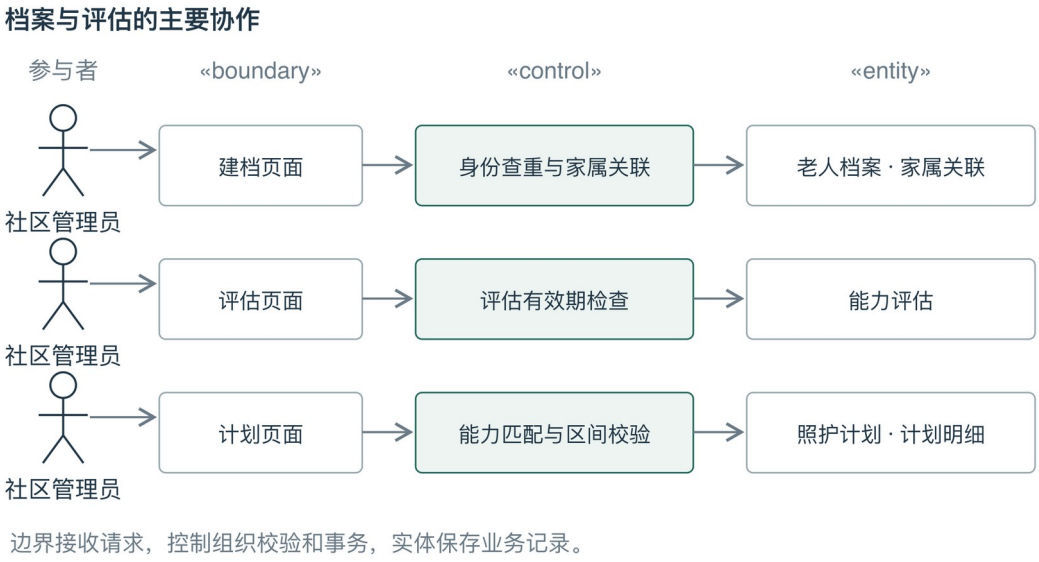


图 2 档案与评估鲁棒图

下单与派单

家属或社区创建预约。管理员按资质、区域、排班和整改状态派单。护工接单、签到、填写记录和签退后进入待确认，家属确认后订单才完成。改派记录保留，失败时恢复原分配。普通评价在服务完成后提交；投诉则经过社区受理、机构或社区回复和家属或社区结案。各步均保存办理人和说明。

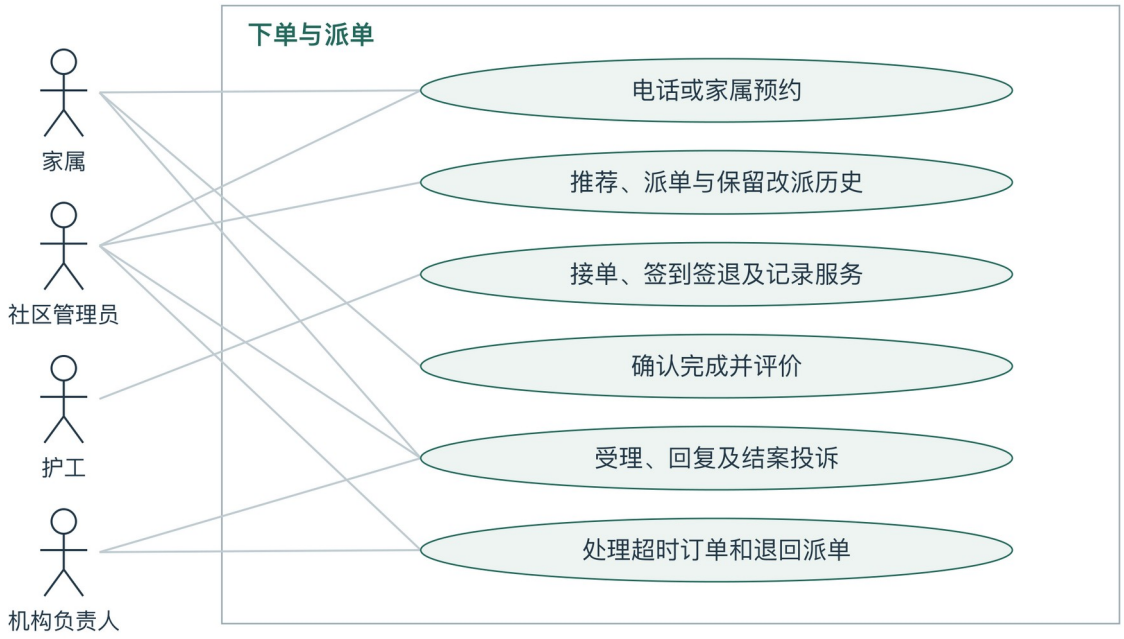


图 3 下单与派单用例图

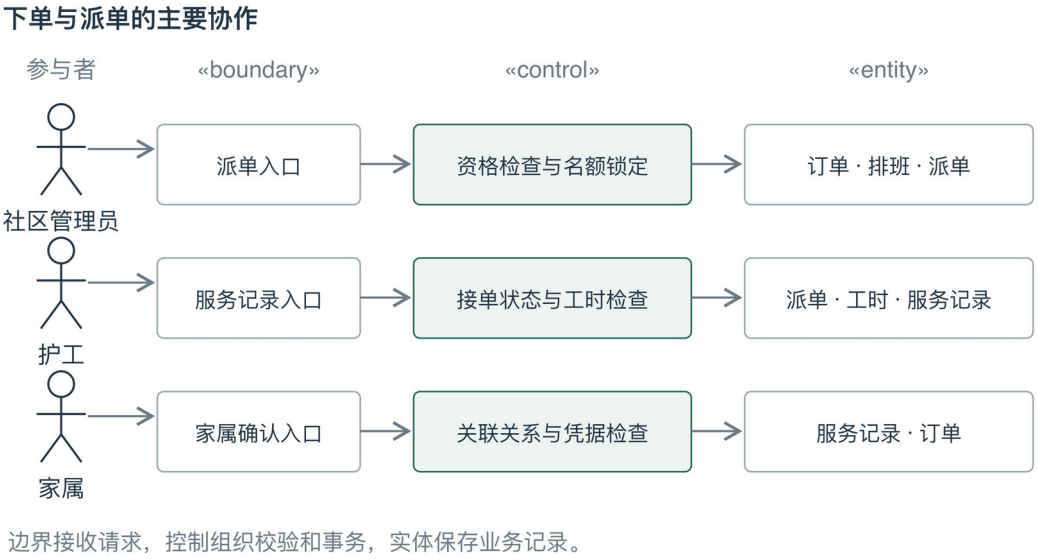


图 4 下单与派单鲁棒图

护工与排班

机构负责人登记招聘申请，录用后建立护工账号，安排岗前培训并登记成绩，再维护项目证书、服务区域与班次。护工服务前后打卡。容量不得超上限，班次和实际预约不得重叠。服务端计算签到签退的围栏距离，围栏、时段或服务时长异常进入人工复核，未经认定的工时不参与补贴。

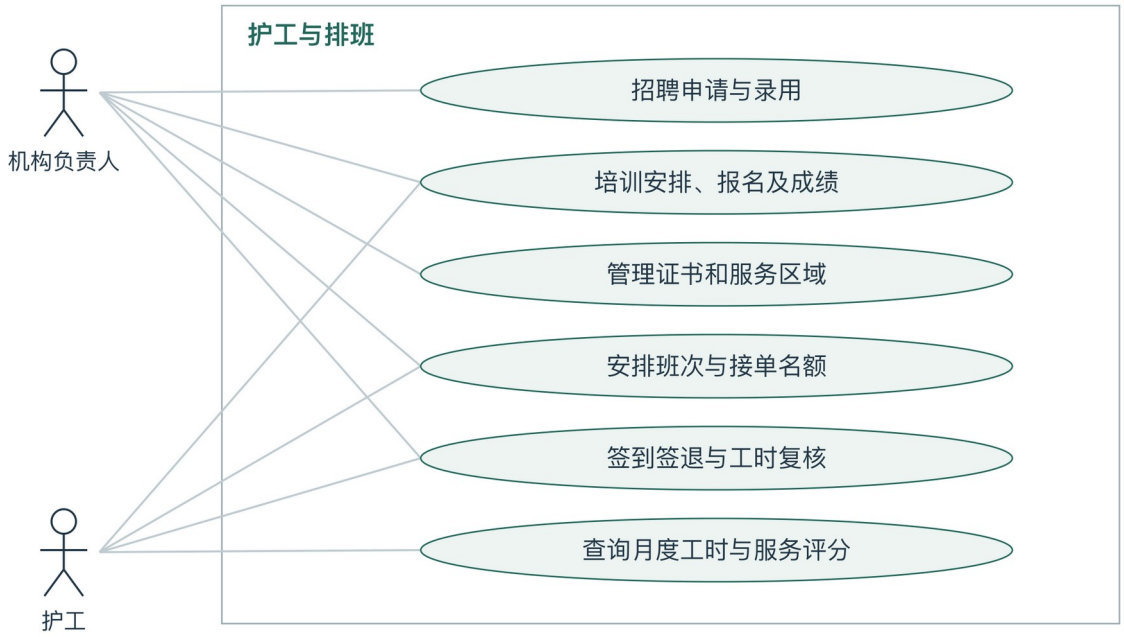


图 5 护工与排班用例图

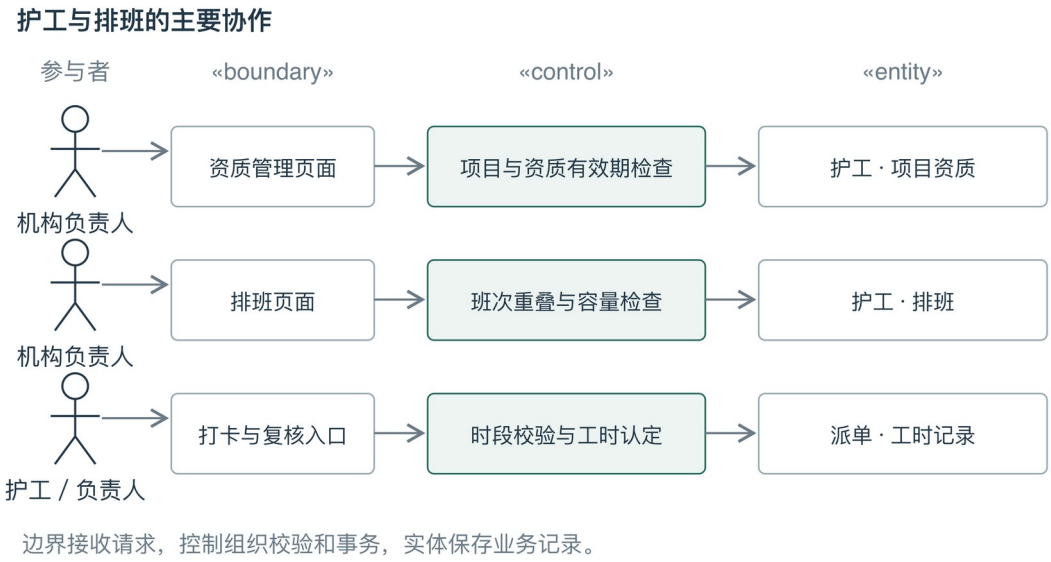


图 6 护工与排班鲁棒图

监测与应急

设备绑定历史保存老人归属和时间区间。跨社区迁移前先解绑原设备，设备本身仍归原社区管理。体征上传先校验采集时绑定，再按课程演示阈值标记异常并建立待确认事件。设备定时模拟采样由管理员明确开启。人员确认异常、响应、登记到场后才能记录处置并关闭；误报单独留存核实原因。未及时响应时由定时任务升级并产生通知。二级及以上事件通知本社区监管人员；其他辖区监管员可按授权查询，不自动订阅全部通知。

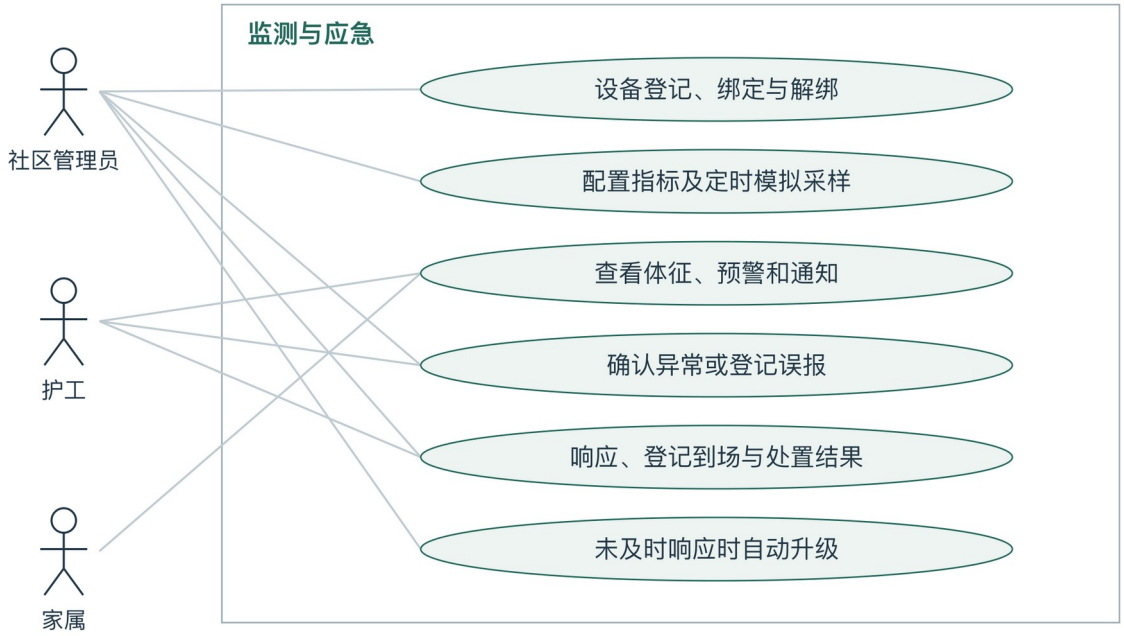


图 7 监测与应急用例图

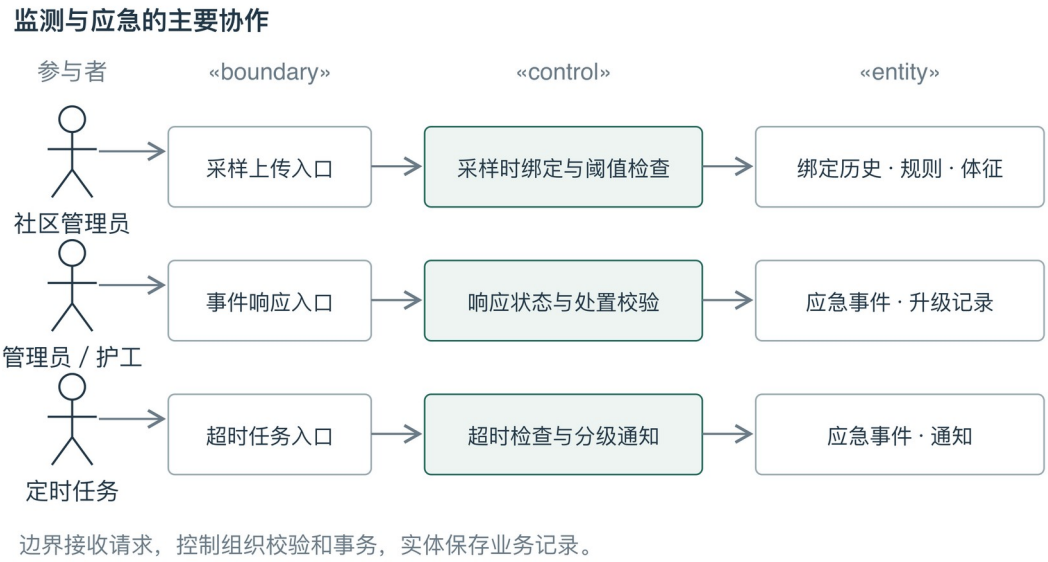


图 8 监测与应急鲁棒图

补贴与监管

社区按政策的月、季或年周期归集可补贴订单。申报事务逐笔检查确认、有效工时、政策和重复资格，任一不合格整体回滚。街道和民政分级审核，再登记模拟拨付；监管按比例建立抽查批次，机构提交整改和考核材料，监管复核通过后发布脱敏公示。未完成整改限制接单。

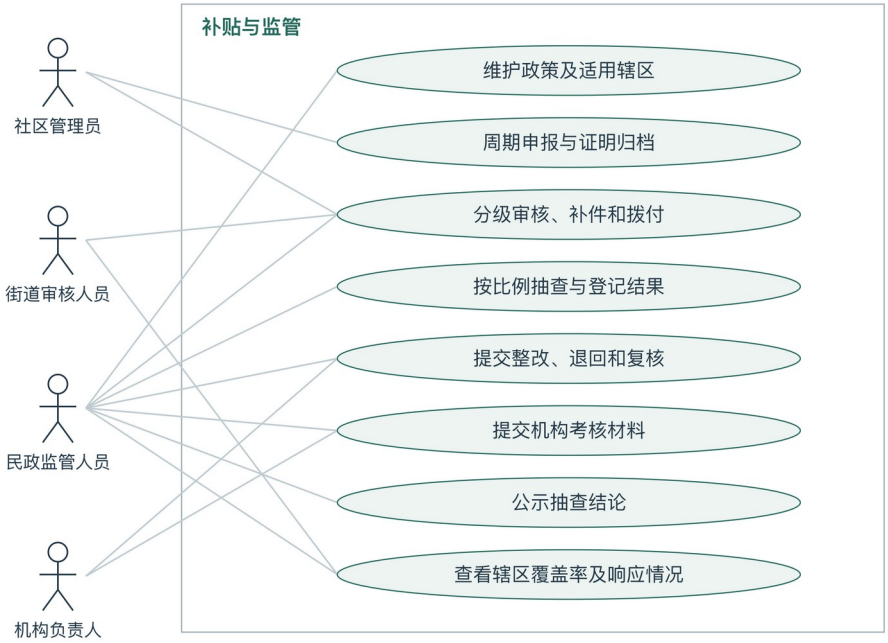


图 9 补贴与监管用例图

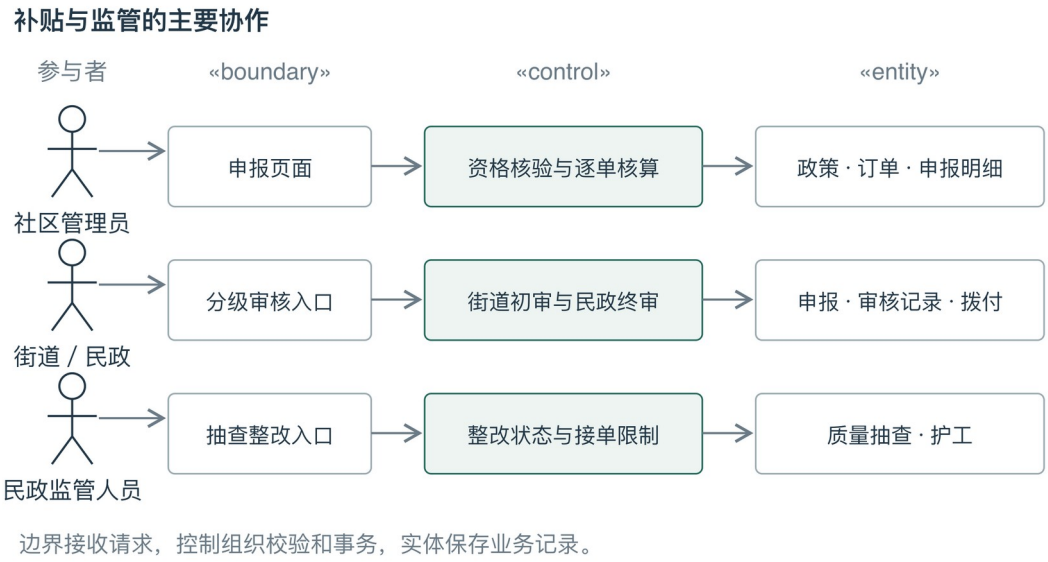


图 10 补贴与监管鲁棒图

投诉办理与整改复核

投诉办理过程

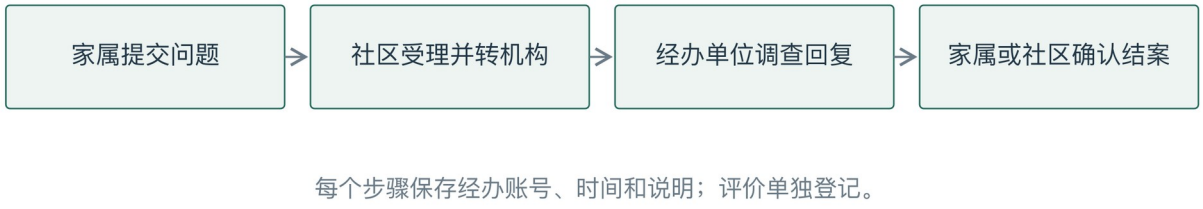


图 11 投诉受理、回复和结案

抽查、整改与公示

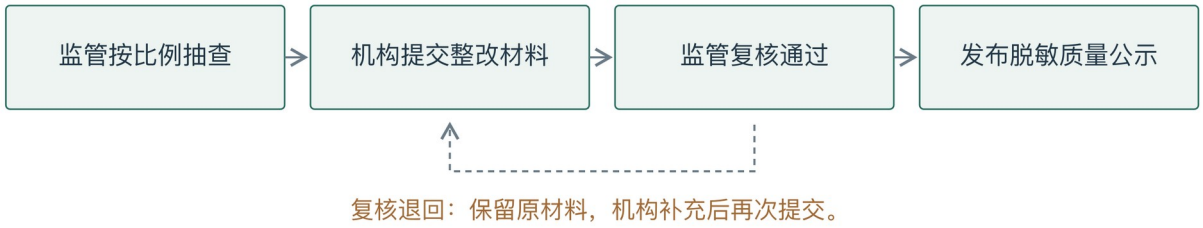


图 12 比例抽查、整改复核与公示

投诉和整改采用独立的办理记录表。经办机构或社区回复后由家属或社区确认结案；整改材料被退回后重新提交，旧材料与审核意见仍可查看。公示从已完成抽查中生成，只保留方式、评分和处置状态。

非功能性需求

系统除了支持正常业务，还要拒绝不符合条件的操作。主外键和唯一约束防止无效关联、重复记录；跨表校验检查计划、资质和订单状态。多人同时操作时，同一老人不能产生重叠计划，同一班次也不能超额接单。查询结果按账号的数据范围过滤，改派、换绑和审核记录保留历史。数据库恢复采用全量备份与 binlog，并在独立数据库中验证。

三、系统概念结构设计

全局模型包含 47 个实体。下图按主要用途归类，方便定位各表；后续局部 UML 图再展开主外键联系。完整 47 实体全局 UML 另附矢量 PDF 和 SVG，可放大查看。局部图展示主键与外键，完整属性和约束见数据字典。PK 表示主键，FK 表示外键；↗ 表示目标实体位于另一张局部图。

全局数据模型分区

47 张基础表按主要用途归类；跨模块通过外键关联。

公共基础 6 张表	community · organization · account account_scope · service_item · audit_log
A 档案与评估 8 张表	elder · elder_tag · guardian · assessment care_bundle · bundle_item · care_plan · plan_item
B 下单与派单 5 张表	service_order · dispatch · service_record complaint · complaint_log
C 护工与排班 8 张表	caregiver · credential · caregiver_area · shift attendance · recruitment · training · training_record
D 监测与应急 8 张表	device · device_binding · metric_rule · vital emergency · emergency_log · notification · event_step
E 补贴与监管 12 张表	policy · policy_area · claim · claim_item · claim_review payment · claim_evidence · inspection · inspection_batch rectification_submission · organization_report · public_notice

关联主线：老人 → 评估 / 计划 → 服务订单 → 派单 / 工时 → 补贴明细

图 13 全局数据模型分区及表清单

公共基础模型

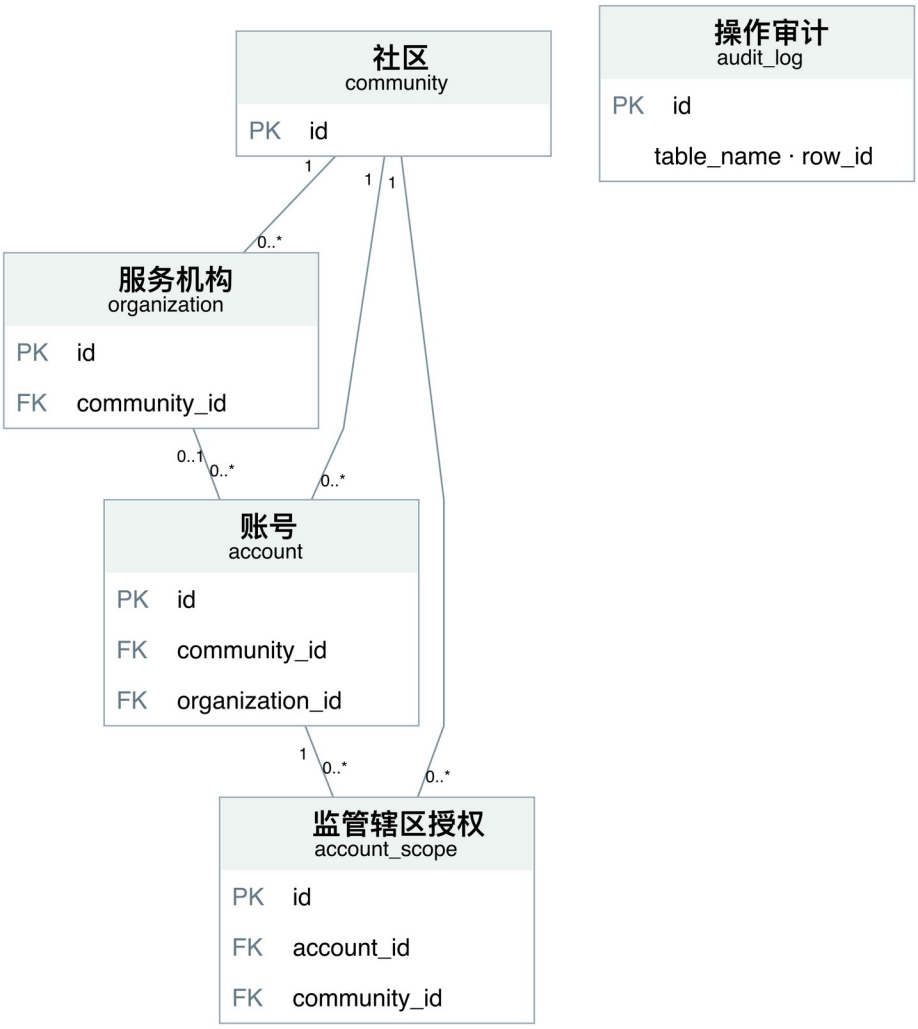


图 14 社区、机构、账号及审计实体

社区、服务机构和账号为业务模块提供归属依据。审计表以表名和记录编号保存变更对象，因此不对各业务表建立直接外键。图中 1 表示必须对应一条父记录，0..1 表示可选且最多一条，0..* 表示可以没有或有多条。

档案与评估局部模型

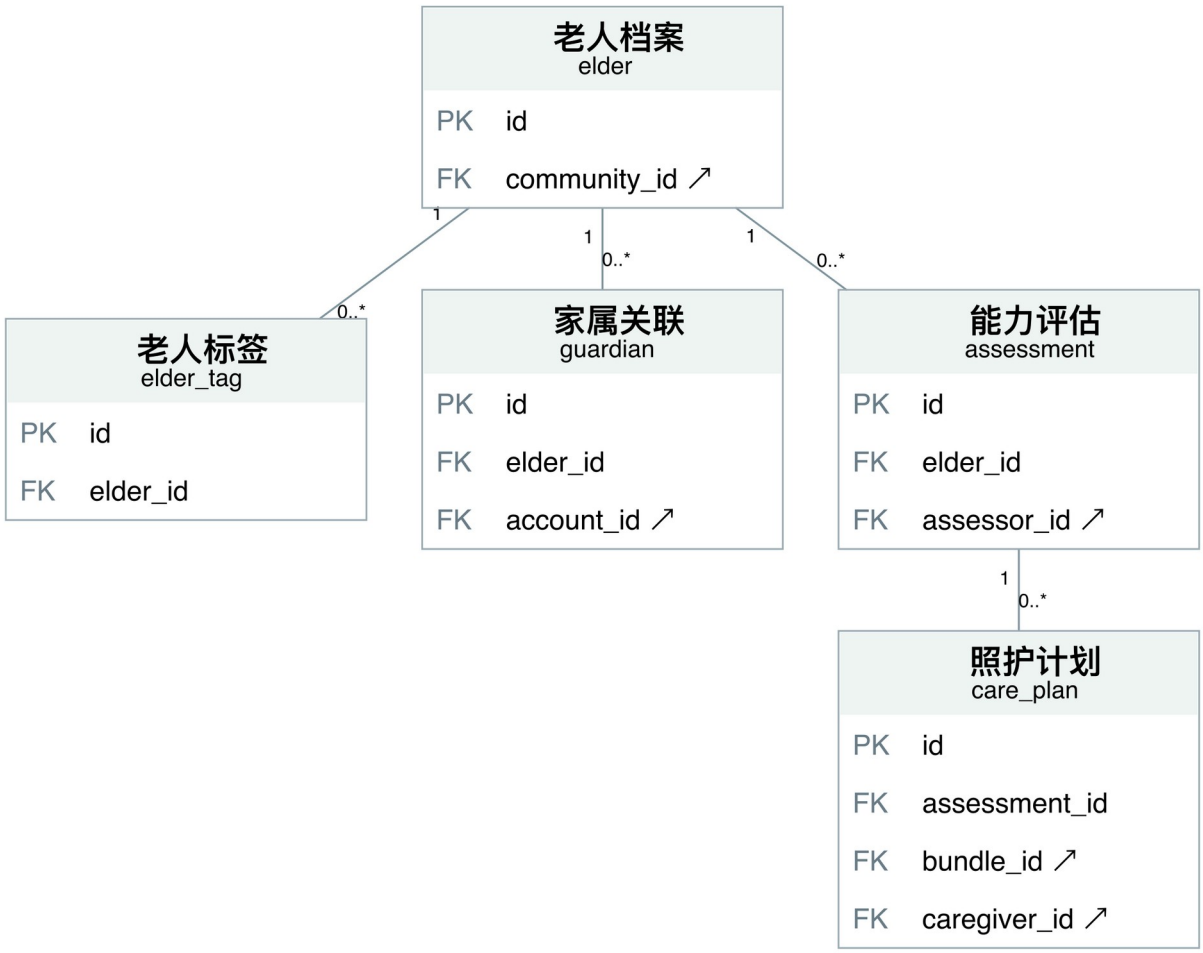


图 15 档案与评估局部 UML 类图

一位老人可以有多次能力评估，每份照护计划引用所采用的评估。家属通过关联表与老人建立联系。

服务包与计划明细

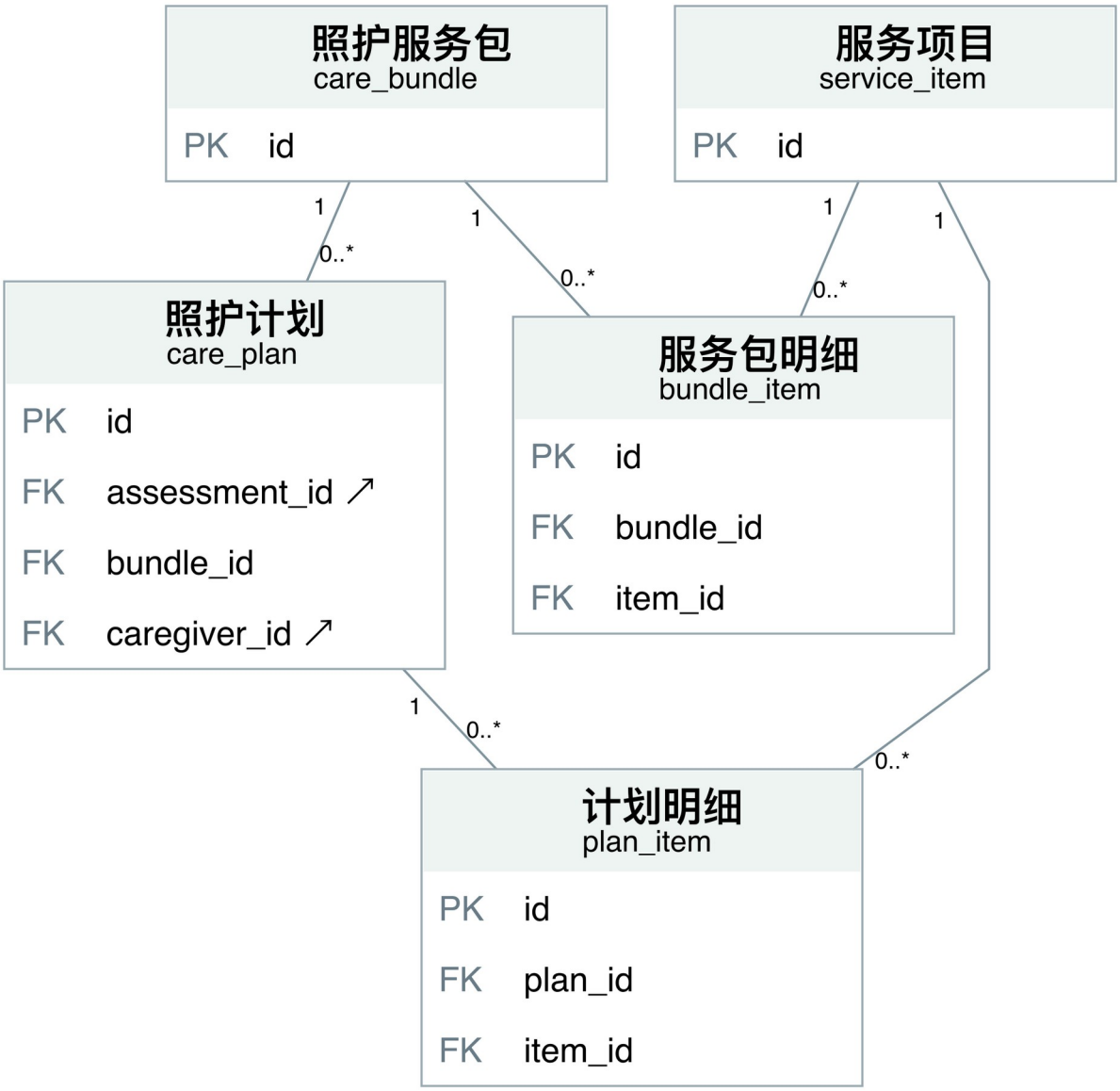


图 16 服务包、项目与计划明细的关系

服务包通过 bundle_item 关联多个服务项目；计划通过 plan_item 保存本次采用的项目与频次。计划引用服务包，同时引用制定时采用的能力评估。

下单与派单局部模型

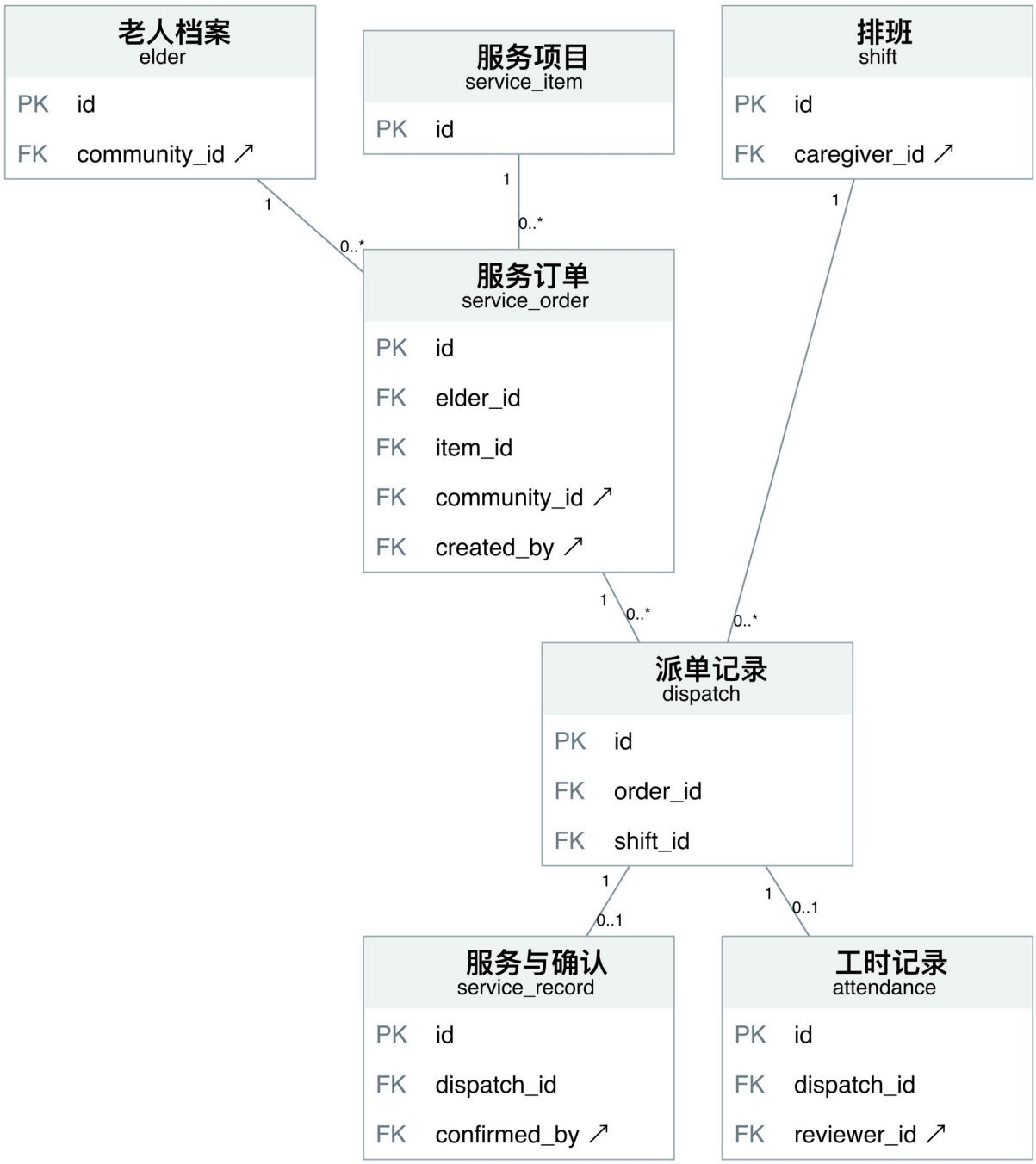


图 17 下单与派单局部 UML 类图

一张订单可以有多条派单记录，改派时保留旧记录。服务记录和签到签退记录引用具体派单。

护工与排班局部模型

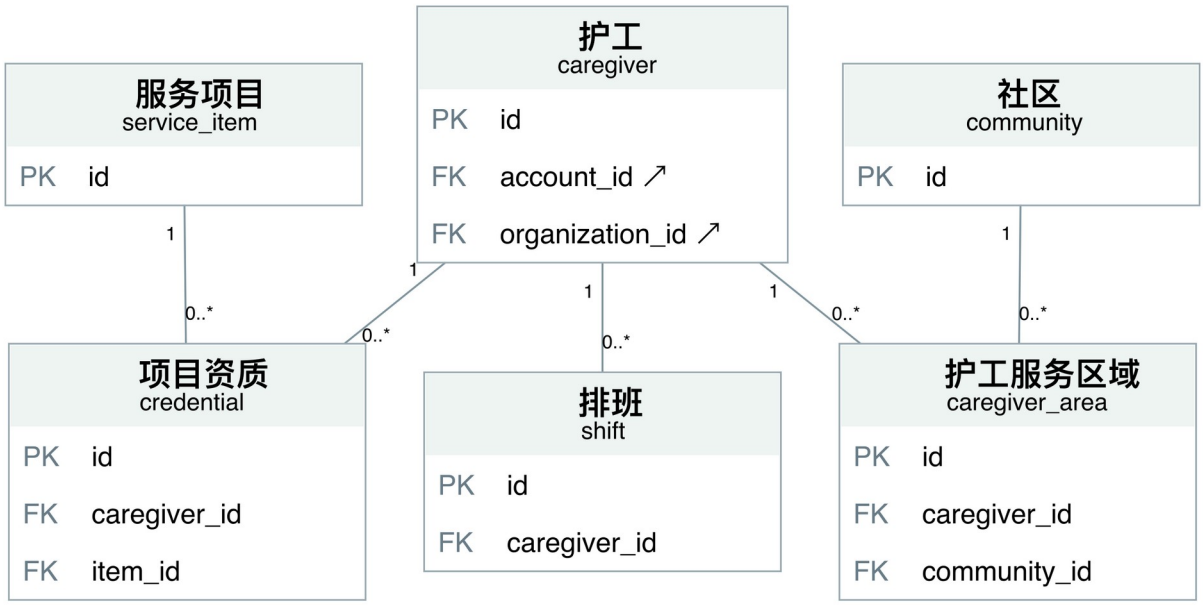


图 18 护工与排班局部 UML 类图

护工通过资质表关联可提供的服务项目，通过排班表保存工作时段。班次记录容量和已占用名额。

监测与应急局部模型

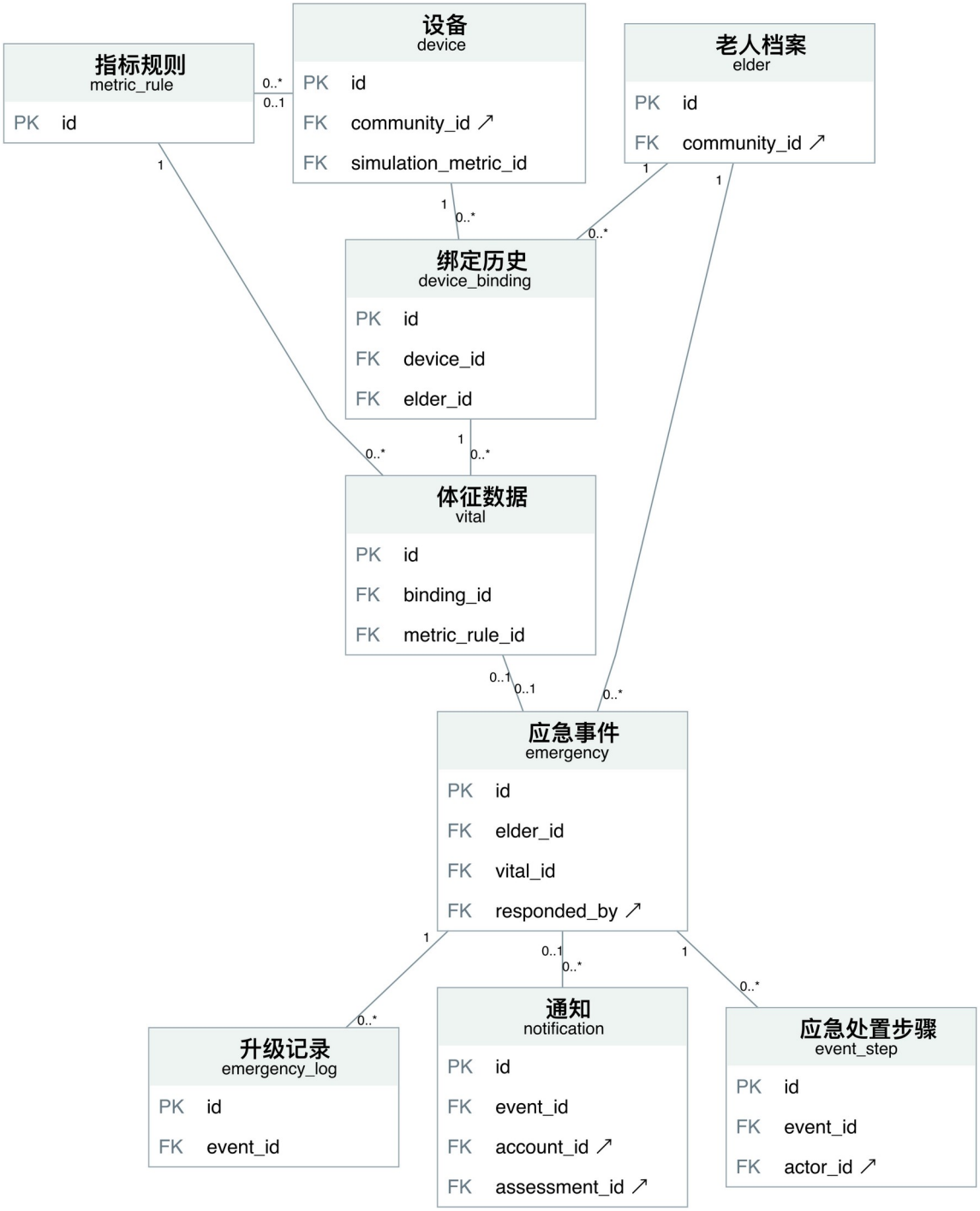


图 19 监测与应急局部 UML 类图

设备与老人通过绑定记录建立联系。体征引用采集时的绑定编号，换绑不会改变旧数据的老人归属。

补贴与监管局部模型

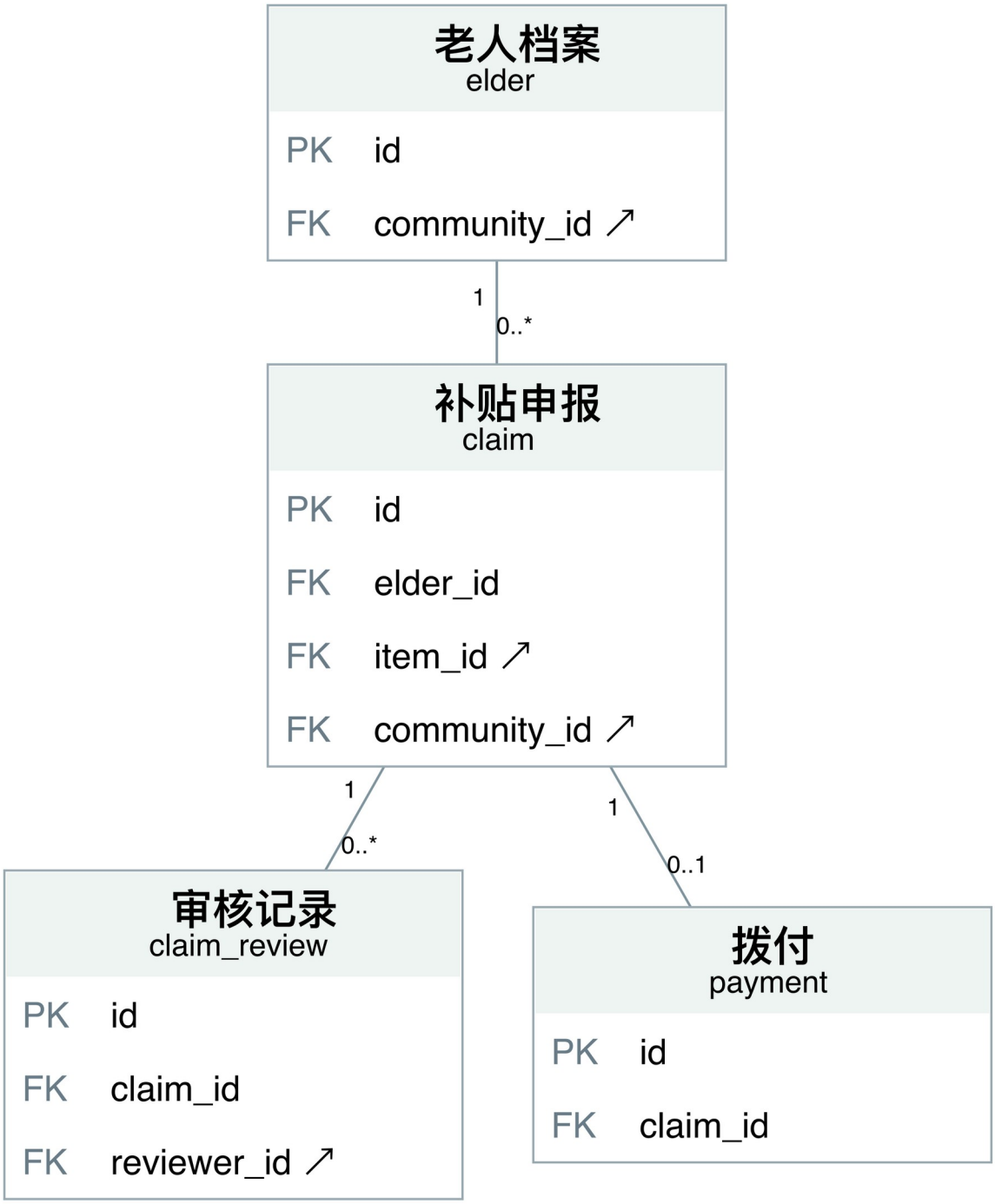


图 20 补贴与监管局部 UML 类图

补贴申报包含订单明细，审核记录和拨付记录分别保存。明细保留核算时的政策、单价和有效工时。

申报明细与核算依据

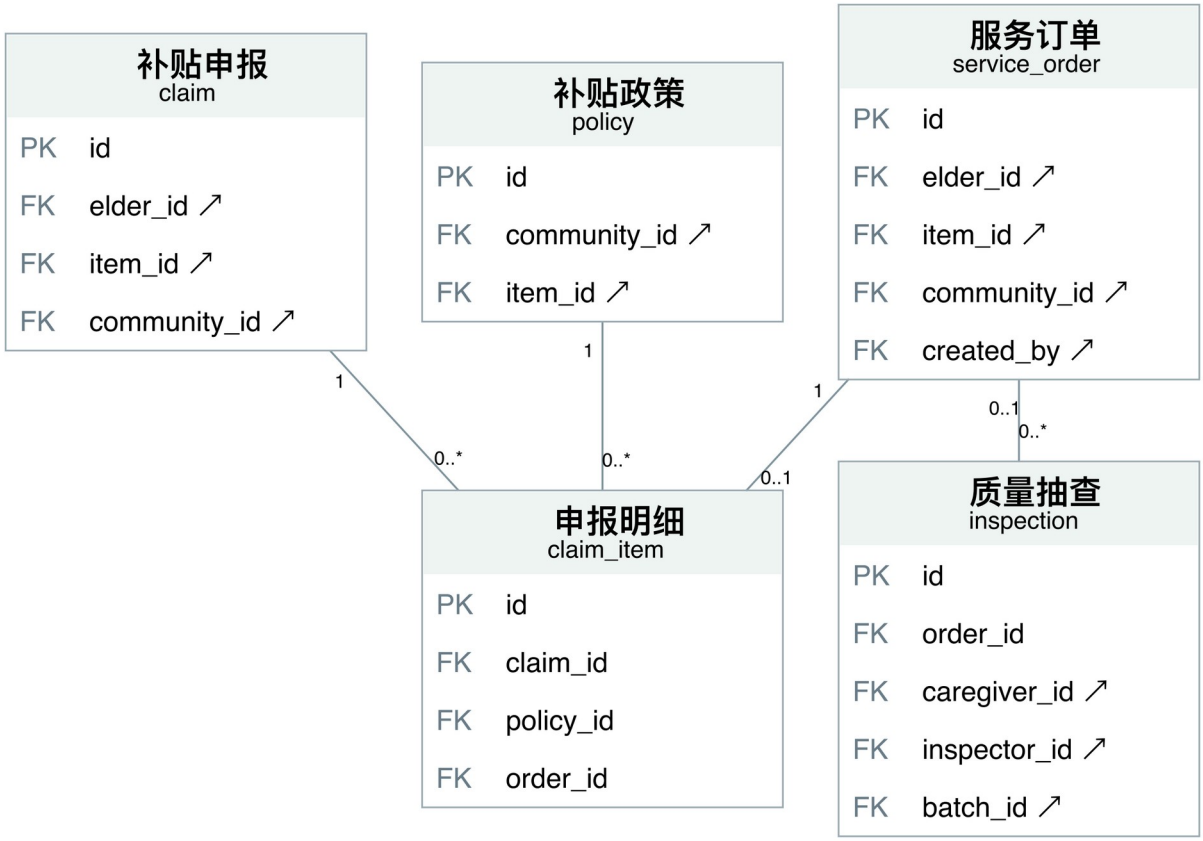


图 21 订单、政策与补贴明细的关系

每条申报明细关联一个订单和一项政策，保存核算单价及有效工时。质量抽查引用订单与护工，抽查结论会影响后续接单资格。

评价投诉记录



图 22 评价投诉记录局部 UML 类图

评价与投诉引用服务订单，办理日志引用反馈记录并保存经办账号。

招聘与岗前培训

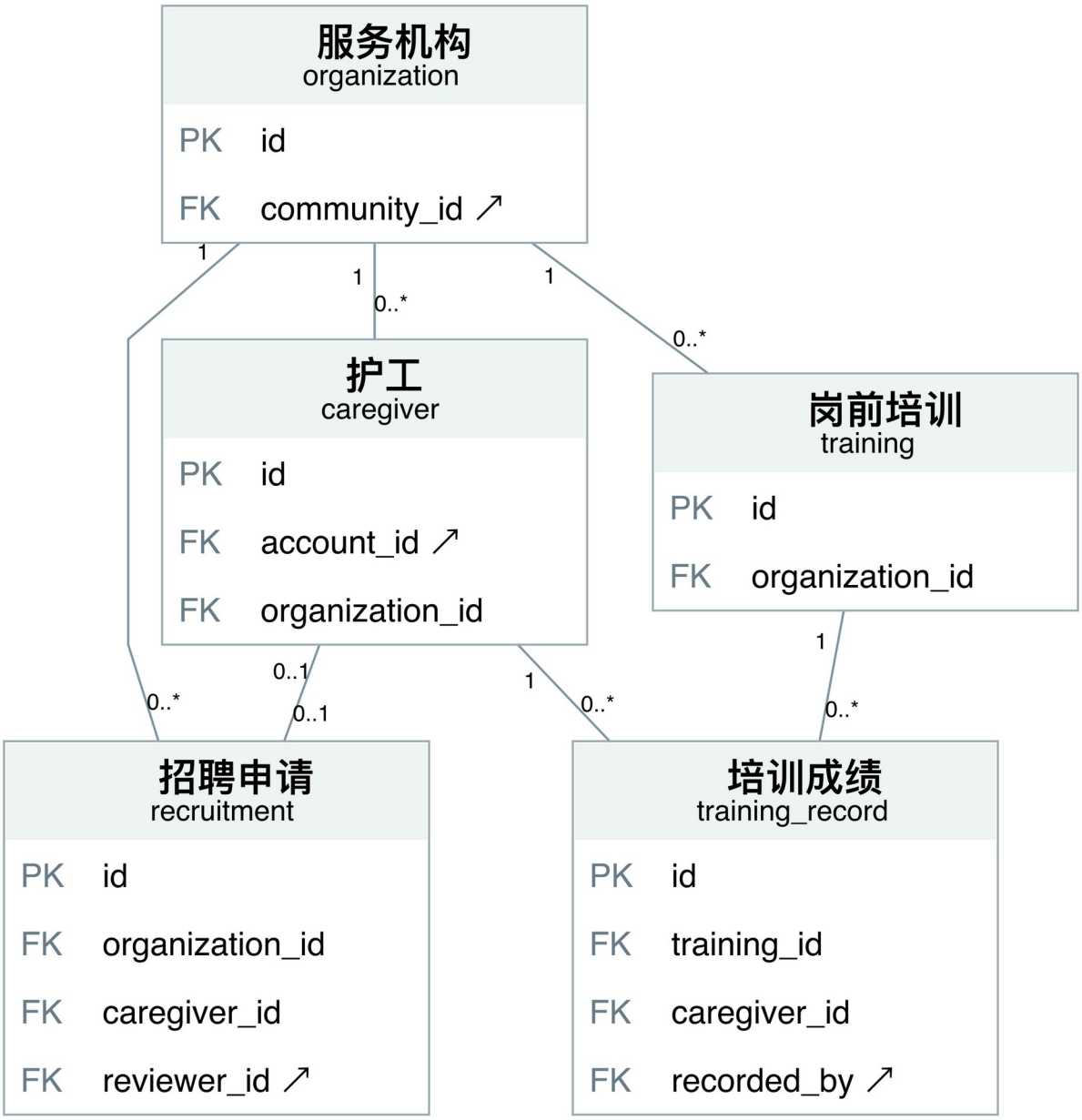


图 23 招聘与岗前培训局部 UML 类图

招聘归属于机构；培训与护工通过参训记录形成多对多联系，成绩只在培训结束后登记。

政策适用范围

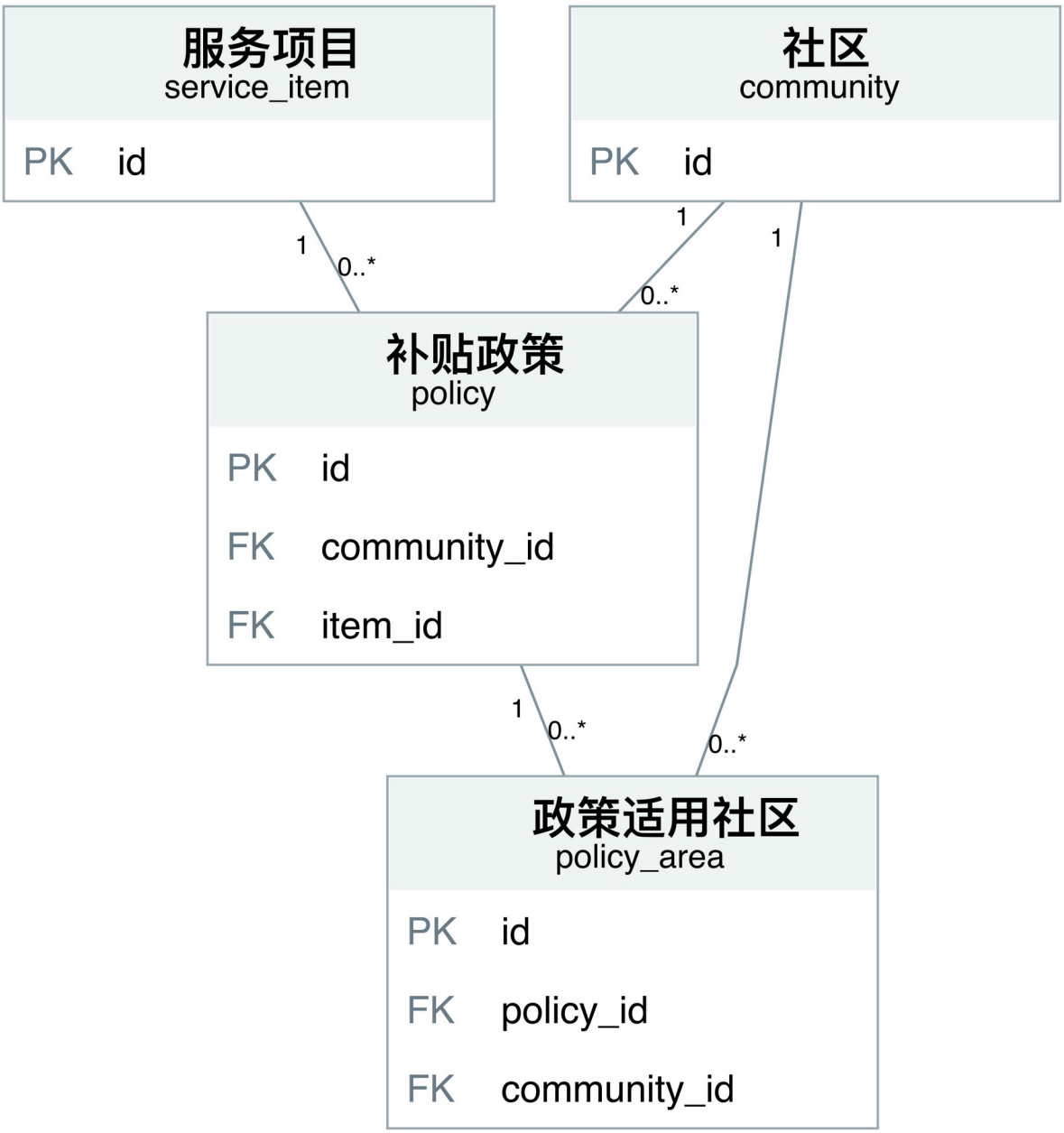


图 24 政策适用范围局部 UML 类图

同一政策可覆盖多个授权社区。适用条件还包括能力等级、年龄和标签，结算周期分别为月、季或年。

抽查与机构材料

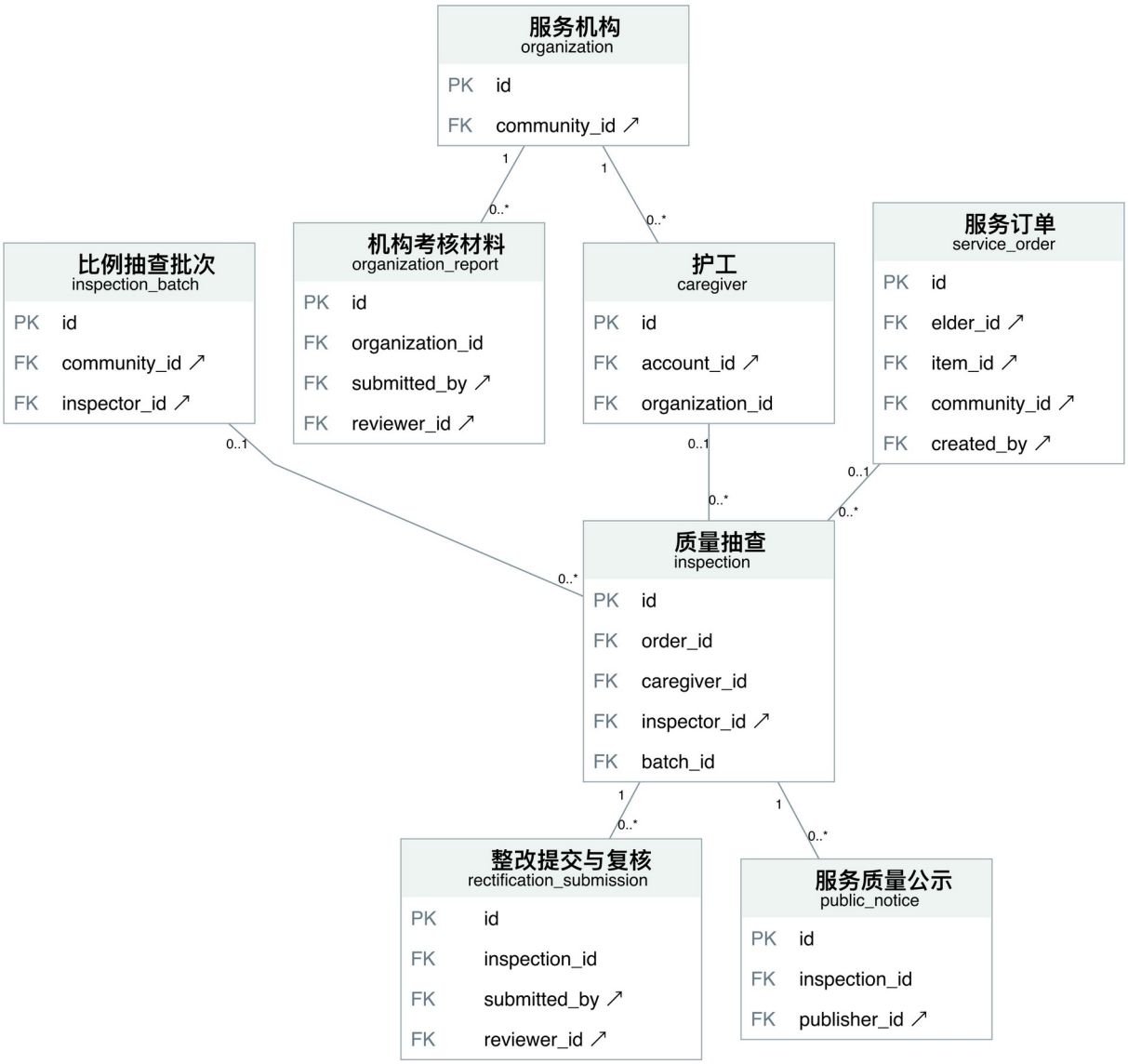


图 25 抽查与机构材料局部 UML 类图

批次保存抽样范围及数量；每条检查明确关联订单或护工，整改提交和公示引用检查记录。机构考核材料单独保留每次提交。

申报证明与审核

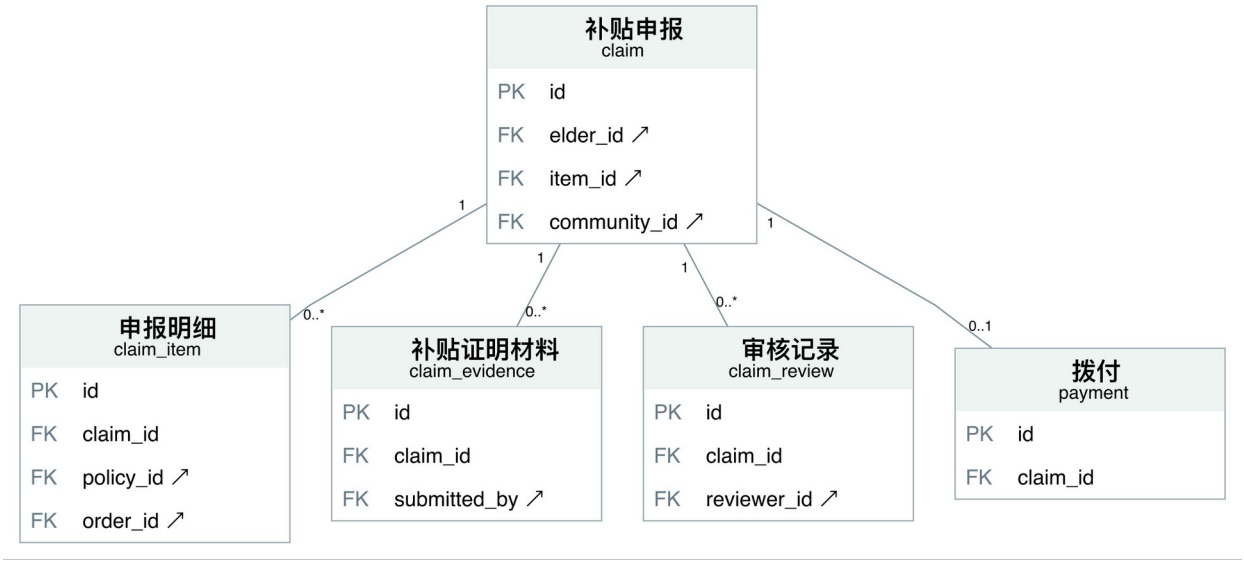


图 26 申报证明与审核局部 UML 类图

证明材料保存文本及内容摘要。驳回后在原申报补件重报，保留明细与历次审核。

四、系统的逻辑结构设计

所有业务实体采用稳定的内部编号作为主键；身份号、设备序列号、证书号等业务唯一键另行约束。多值属性拆成关联表，实体联系通过外键表达。完整关系模式与所有字段见《数据字典与关系模式》。

范式判断方法

对每个关系先列出主键与其他候选键，再检查非主属性是否完全依赖候选键，并排除非键属性间的传递依赖。单列代理主键只能帮助标识记录，不能自动证明第三范式。

实体关系

community、organization、account、elder、assessment、service_item、care_bundle、caregiver、credential、shift、device、metric_rule、emergency、policy、inspection、complaint、audit_log 均以 id 确定本行事实。姓名不决定住址、机构或身份；评估得分与等级的对应口径随量表和版本而变化，本系统保存评估结论，不把 score 当作独立决定等级的全局函数。account 中 community_id 和 organization_id 表示授权范围，并非根据账号角色推导出的机构归属。

elder_tag 的候选键为 (elder_id,tag)；guardian 为 (elder_id,account_id)；bundle_item 为 (bundle_id,item_id)；caregiver_area 为 (caregiver_id,community_id)；plan_item 为 (plan_id,item_id)。各关联属性依赖完整组合键。外键指向对象的名称、地址等不重复放入关联表。

care_plan 只保存 assessment_id，老人归属从 assessment 派生；因此消除了 assessment_id → elder_id 在计划表内形成的传递依赖。v_plan_context 仅为查询展开归属，不新增事实。dispatch 只保存 shift_id，护工归属从 shift 派生，避免 shift_id → caregiver_id 的重复存储。v_dispatch_context 同理。

service_record 和 attendance 的 dispatch_id 是备用候选键；payment 的 claim_id 和 reference_no 均唯一；device 的 serial_no、credential 的 certificate_no、caregiver 的 account_id、elder 的 identity_hash、account 的 username 是业务候选键。

claim 的业务候选键是 (elder_id,item_id,period)。政策引用位于 claim_item 中，不在 claim 同时保存 policy_id 与可由它决定的 item_id。claim_item 的 order_id 唯一，确保同一服务订单不能重复获补；采用的政策、核定分钟和补贴单价属于该次核算事实。

补充关系的函数依赖

account_scope 的候选键为 (account_id,community_id)，policy_area 为 (policy_id,community_id)，用于表达多社区授权和政策适用范围；不能再把监管账号的单一归属社区当作全部权限。training_record 的候选键为 (training_id,caregiver_id)，成绩由完整参训关系确定。recruitment 的录用护工编号可作为非空记录的业务唯一键，一位应聘人是否录用取决于申请处理结果。

complaint_log、event_step、claim_evidence 以各自 id 确定一次办理或材料提交事实。rectification_submission 与 organization_report 分开保存每次提交和对应复核，退回后新增提交，不覆盖旧材料。inspection_batch 保存抽查范围、总体数量、比例、种子和样本数；inspection.batch_id 标识实际抽中的记录。public_notice 从已完成抽查提取脱敏内容，保留发布与撤回信息。

历史事实与物理派生列

service_order 的地址、服务社区和金额是下单时的事实快照，并不由老人当前地址或目录当前指导价永久决定。claim_item 的 rate_snapshot 保存核算时价格，政策未来调整不能改变既有申报。device_binding 记录时间区间，vital 引用采集时的绑定，不由设备当前归属反推。

dispatch.active_order 和 device_binding.active_device 是物理生成列，用于条件唯一性约束；它们不属于独立的概念属性。shift.used_count、claim.amount 为受控物理汇总值；逻辑上可由明细推导，保留是为了容量并发控制和核算校验，必须通过事务同步或触发器核对。这些物理实现选择应与纯逻辑关系模式区别说明，不能把所有缓存都当作独立原始事实。

完整性与时间约束

主键、外键、UNIQUE 和 CHECK 约束负责基本合法性。计划和排班采用左闭右开区间，用 start1 < end2 且 end1 > start2 判定重叠。assert_plan 与 assert_shift 先锁定同一老人或护工再检查区间，应用事务使用 READ COMMITTED。派单先锁订单和护工，然后检查资质、区域、整改和预约冲突，并原子占用班次名额。事务失败回滚改派与名额变化。

历史禁止删除触发器保护评估、派单、服务记录、工时、体征、申报明细、审核、拨付和审计。已确认服务与已支付申报禁止普通修改。数据库管理员仍可管理结构与授权，本项目不声称能抵御拥有最高数据库权限的攻击者。

关系模式清单

社区：community(id [PK], name, district, area_code, street_name, center_x, center_y, radius_m, fence_m)。

服务机构：organization(id [PK], name, community_id [FK])。

账号：account(id [PK], username, password_hash, role, community_id [FK], organization_id [FK], failures, locked_until, active)。

老人档案：elder(id [PK], name, identity_hash, identity_cipher, birth_date, address, community_id [FK], status, position_x, position_y)。

老人标签：elder_tag(id [PK], elder_id [FK], tag)。

家属关联：guardian(id [PK], elder_id [FK], account_id [FK], relation_name, phone_cipher, active)。

能力评估：assessment(id [PK], elder_id [FK], scale_name, score, ability_level, assessor_id [FK], assessed_at, valid_until)。

服务项目：service_item(id [PK], name, category, unit, price, duration_minutes, subsidizable)。

照护服务包：care_bundle(id [PK], name, ability_level)。

服务包明细：bundle_item(id [PK], bundle_id [FK], item_id [FK], weekly_frequency)。

护工：caregiver(id [PK], account_id [FK], organization_id [FK], name, status)。

项目资质：credential(id [PK], caregiver_id [FK], item_id [FK], certificate_no, certificate_level, valid_from, valid_until)。

护工服务区域：caregiver_area(id [PK], caregiver_id [FK], community_id [FK])。

照护计划：care_plan(id [PK], assessment_id [FK], bundle_id [FK], caregiver_id [FK], starts_at, ends_at, status)。

计划明细：plan_item(id [PK], plan_id [FK], item_id [FK], weekly_frequency)。

排班：shift(id [PK], caregiver_id [FK], starts_at, ends_at, capacity, used_count, status)。

服务订单：service_order(id [PK], elder_id [FK], item_id [FK], community_id [FK], starts_at, ends_at, address, channel, amount, status, created_by [FK], created_at, position_x, position_y)。

派单记录：dispatch(id [PK], order_id [FK], shift_id [FK], status, reason, assigned_at, accepted_at, active_order)。

服务与确认：service_record(id [PK], dispatch_id [FK], notes, confirmed_by [FK], confirmed_at, confirmation)。

工时记录：attendance(id [PK], dispatch_id [FK], check_in, check_out, location_note, fence_ok, status, effective_minutes, reviewer_id [FK], review_reason, checkin_x, checkin_y, checkout_x, checkout_y, checkout_fence_ok)。

设备：device(id [PK], serial_no, type_name, location_note, status, community_id [FK], simulation_enabled, simulation_metric_id [FK], simulation_seconds, last_simulated_at)。

绑定历史：device_binding(id [PK], device_id [FK], elder_id [FK], bound_at, unbound_at, active_device)。

指标规则：metric_rule(id [PK], metric, low_value, high_value, response_seconds)。

体征数据：vital(id [PK], binding_id [FK], metric_rule_id [FK], sample_key, collected_at, metric_value, abnormal, source)。

应急事件：emergency(id [PK], elder_id [FK], vital_id [FK], source, level, occurred_at, deadline, status, responded_by [FK], responded_at, arrived_at, result, initial_deadline, closed_at, close_reason)。

升级记录：emergency_log(id [PK], event_id [FK], level, created_at, message)。

通知：notification(id [PK], event_id [FK], account_id [FK], message, created_at, assessment_id [FK], dedupe_key)。

补贴政策：policy(id [PK], name, community_id [FK], item_id [FK], ability_level, rate, starts_at, ends_at, settlement_cycle, minimum_age, required_tag)。

补贴申报：claim(id [PK], elder_id [FK], item_id [FK], period, amount, status, community_id [FK], period_start, period_end)。

申报明细：claim_item(id [PK], claim_id [FK], policy_id [FK], order_id [FK], amount, effective_minutes, rate_snapshot)。

审核记录：claim_review(id [PK], claim_id [FK], reviewer_id [FK], stage, decision, reason, created_at)。

拨付：payment(id [PK], claim_id [FK], reference_no, amount, paid_at)。

质量抽查：inspection(id [PK], order_id [FK], caregiver_id [FK], inspector_id [FK], method, score, issue, rectification, status, result, batch_id [FK], created_at)。

评价投诉：complaint(id [PK], order_id [FK], account_id [FK], rating, content, created_at, status, organization_id [FK], handler_id [FK], result, updated_at, kind)。

操作审计：audit_log(id [PK], table_name, row_id, operation, actor, old_status, new_status, created_at, before_data, after_data)。

监管辖区授权：account_scope(id [PK], account_id [FK], community_id [FK])。

政策适用社区：policy_area(id [PK], policy_id [FK], community_id [FK])。

招聘申请：recruitment(id [PK], organization_id [FK], candidate_name, phone_cipher, status, caregiver_id [FK], reviewer_id [FK], review_note, created_at, reviewed_at)。

岗前培训：training(id [PK], organization_id [FK], title, starts_at, ends_at)。

培训成绩：training_record(id [PK], training_id [FK], caregiver_id [FK], result, score, recorded_by [FK], recorded_at)。

投诉办理记录：complaint_log(id [PK], complaint_id [FK], actor_id [FK], status, message, created_at)。

比例抽查批次：inspection_batch(id [PK], community_id [FK], inspector_id [FK], target_type, starts_at, ends_at, ratio, population, sample_count, sample_seed, created_at)。

整改提交与复核：rectification_submission(id [PK], inspection_id [FK], submitted_by [FK], measures, evidence, created_at, decision, reviewer_id [FK], review_note, reviewed_at)。

机构考核材料：organization_report(id [PK], organization_id [FK], period, summary, evidence, submitted_by [FK], status, reviewer_id [FK], review_note, created_at, reviewed_at)。

服务质量公示：public_notice(id [PK], inspection_id [FK], publisher_id [FK], title, body, published_at, withdrawn_at, withdraw_reason)。

补贴证明材料：claim_evidence(id [PK], claim_id [FK], submitted_by [FK], name, content, content_sha256, created_at)。

应急处置步骤：event_step(id [PK], event_id [FK], actor_id [FK], action, message, created_at)。

所有外键及其目标列单列在 evidence/foreign-keys.tsv 与数据字典中；模式中的生成列仅服务于物理唯一性约束。

五、系统物理结构设计、持久层设计和关键业务代码

运行环境与分层

程序在 macOS arm64 环境运行，使用 Java 17.0.15、Spring Boot 3.4.4、Spring JDBC 和 MySQL 8.4.11。浏览器提交请求后，由接口层交给业务方法处理，再通过 DAO 访问数据库。Windows 版本提供 JAR、数据库初始化脚本和启动脚本，不依赖 Docker。

【待验证】在 Windows 目标电脑上完成初始化、启动和主要业务操作检查。

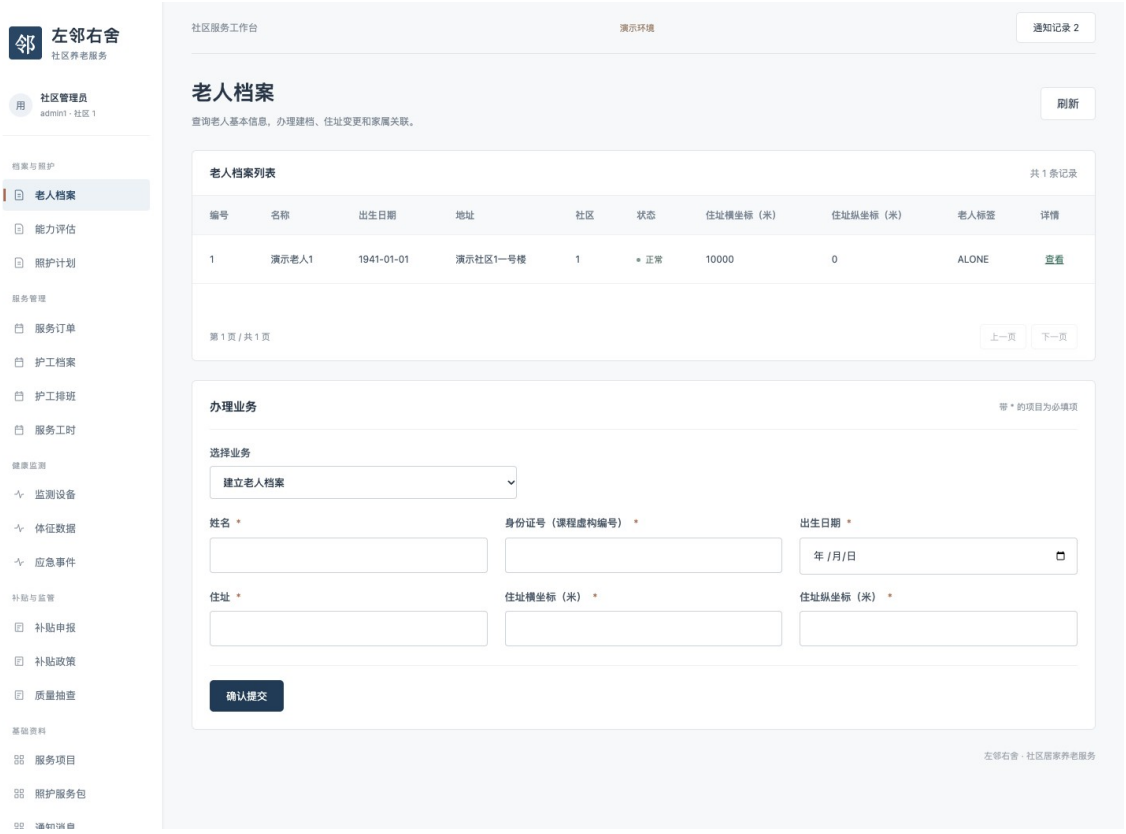


图 27 实际浏览器登录后的社区工作台

DAO 基础实现

```
package edu.bjfu.eldercare;

import org.springframework.jdbc.core.JdbcTemplate;

import org.springframework.jdbc.support.GeneratedKeyHolder;

import org.springframework.stereotype.Repository;

import java.sql.Statement;
```

```
import java.util.*;

@Repository

public class Store {

 public final JdbcTemplate jdbc;

 public Store(JdbcTemplate jdbc){this.jdbc=jdbc;}

 public List<Map<String,Object>> rows(String sql,Object...args){return
jdbc.queryForList(sql,args);}

 public Map<String,Object> one(String sql,Object...args){var
r=rows(sql,args);if(r.isEmpty())throw new Rule("记录不存在");return r.get(0);}

 public long insert(String sql,Object...args){var k=new
GeneratedKeyHolder();jdbc.update(c->{var
p=c.prepareStatement(sql,Statement.RETURN_GENERATED_KEYS);for(int
i=0;i<args.length;i++)p.setObject(i+1,args[i]);return p;},k);return
Objects.requireNonNull(k.getKey()).longValue();}

 public int update(String sql,Object...args){return jdbc.update(sql,args);}

 public long count(String sql,Object...args){return
Objects.requireNonNull(jdbc.queryForObject(sql,Long.class,args));}

 public void call(String name,Object...args){if(!
Set.of("assert_order","assert_worker_order","assign_order","verify_hours","submit_claim","e
xpire_plans","escalate_events","remind_assessments").contains(name))throw new Rule("非
法过程");jdbc.update("CALL "+name+"("+String.join(",",Collections.nCopies(args.length,"?"))
+")",args);}

 public static long id(Map<String,Object> m,String k){return
((Number)m.get(k)).longValue();}

}

class Rule extends RuntimeException {Rule(String m){super(m);}}
```

SQL 的用户值全部参数绑定。插入方法返回生成主键，查询和更新统一经过

JdbcTemplate。分页入口校验页码与大小并使用固定数据类别白名单。体征批量上传在同

一事务中逐条校验和写入，任一条越权或无效整体回滚。业务方法使用 @Transactional，过程不提前提交。

派单与改派的事务设计

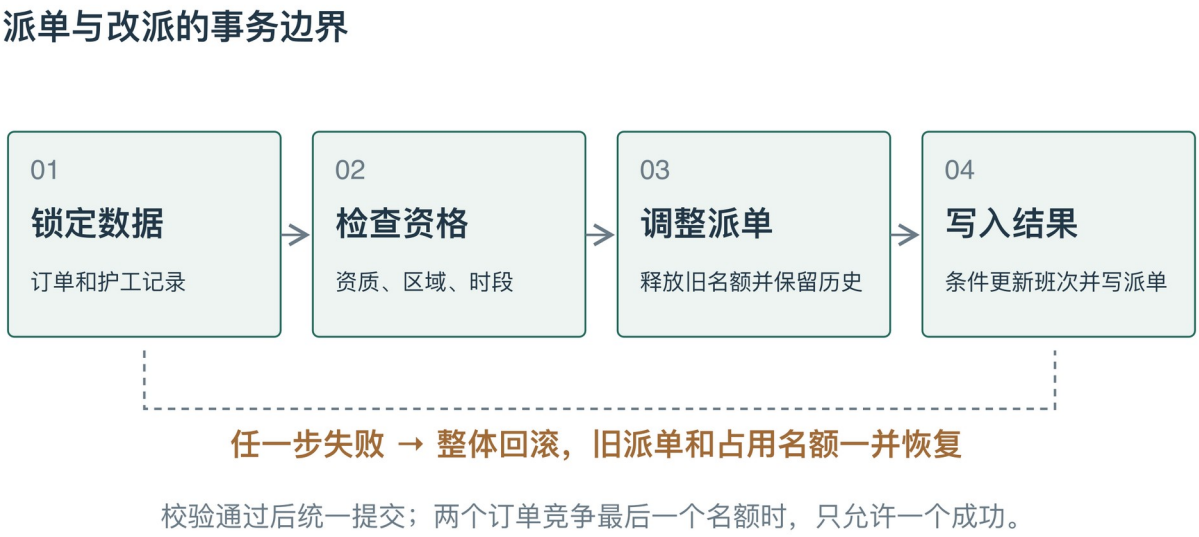


图 28 派单事务及失败回滚

派单过程先锁定订单和护工记录，再检查项目资质、服务区域及预约时段。改派时释放旧名额并保留旧派单，随后通过条件更新检查班次容量并写入新派单。各步在同一事务中完成，后续检查失败时恢复原有分配，避免派单与名额不一致。

完整建表与索引语句

社区

```
CREATE TABLE community(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(80) NOT NULL,
  district VARCHAR(80) NOT NULL,
  area_code VARCHAR(30) NOT NULL UNIQUE);
```

服务机构

```
CREATE TABLE organization(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
```

```
  community_id BIGINT NOT NULL,
  FOREIGN KEY(community_id) REFERENCES community(id));
```

账号

```
CREATE TABLE account(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) NOT NULL UNIQUE,
  password_hash VARCHAR(100) NOT NULL,
  role ENUM('ADMIN','FAMILY','CAREGIVER','MANAGER','STREET','REGULATOR') NOT NULL,
  community_id BIGINT NOT NULL,
  organization_id BIGINT,
  failures INT NOT NULL DEFAULT 0,
  locked_until DATETIME,
  active BOOLEAN NOT NULL DEFAULT TRUE,
  FOREIGN KEY(community_id) REFERENCES community(id),
  FOREIGN KEY(organization_id) REFERENCES organization(id));
```

老人档案

```
CREATE TABLE elder(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(60) NOT NULL,
  identity_hash CHAR(64) NOT NULL UNIQUE,
  identity_cipher TEXT NOT NULL,
  birth_date DATE NOT NULL,
  address VARCHAR(200) NOT NULL,
  community_id BIGINT NOT NULL,
  status ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
  FOREIGN KEY(community_id) REFERENCES community(id));
```

老人标签

```
CREATE TABLE elder_tag(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  elder_id BIGINT NOT NULL,
  tag ENUM('ALONE','DISABLED','LOW_INCOME') NOT NULL,
  UNIQUE(elder_id,tag),
  FOREIGN KEY(elder_id) REFERENCES elder(id));
```

家属关联

```
CREATE TABLE guardian(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  elder_id BIGINT NOT NULL,
  account_id BIGINT NOT NULL,
  relation_name VARCHAR(30) NOT NULL,
  phone_cipher TEXT NOT NULL,
  UNIQUE(elder_id,account_id),
  FOREIGN KEY(elder_id) REFERENCES elder(id),
  FOREIGN KEY(account_id) REFERENCES account(id));
```

能力评估

```
CREATE TABLE assessment(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  elder_id BIGINT NOT NULL,
  scale_name VARCHAR(50) NOT NULL,
  score INT NOT NULL,
  ability_level ENUM('SELF','PARTIAL','SEVERE') NOT NULL,
  assessor_id BIGINT NOT NULL,
  assessed_at DATETIME NOT NULL,
  valid_until DATETIME NOT NULL,
  CHECK(score BETWEEN 0 AND 100),
  CHECK(valid_until>assessed_at),
```

```
  FOREIGN KEY(elder_id) REFERENCES elder(id),
  FOREIGN KEY(assessor_id) REFERENCES account(id),
  INDEX idx_assessment_elder(elder_id,valid_until));
```

服务项目

```
CREATE TABLE service_item(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(80) NOT NULL,
  category VARCHAR(30) NOT NULL,
  unit ENUM('VISIT','HOUR') NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  duration_minutes INT NOT NULL,
  subsidizable BOOLEAN NOT NULL,
  CHECK(price>=0),
  CHECK(duration_minutes>0));
```

照护服务包

```
CREATE TABLE care_bundle(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(80) NOT NULL,
  ability_level ENUM('SELF','PARTIAL','SEVERE') NOT NULL);
```

服务包明细

```
CREATE TABLE bundle_item(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  bundle_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  weekly_frequency INT NOT NULL,
  CHECK(weekly_frequency>0),
  UNIQUE(bundle_id,item_id),
  FOREIGN KEY(bundle_id) REFERENCES care_bundle(id),
  FOREIGN KEY(item_id) REFERENCES service_item(id));
```

护工

```
CREATE TABLE caregiver(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  account_id BIGINT NOT NULL UNIQUE,
  organization_id BIGINT NOT NULL,
  name VARCHAR(60) NOT NULL,
  status ENUM('ACTIVE','LEAVE','LEFT') NOT NULL DEFAULT 'ACTIVE',
  FOREIGN KEY(account_id) REFERENCES account(id),
  FOREIGN KEY(organization_id) REFERENCES organization(id));
```

项目资质

```
CREATE TABLE credential(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  caregiver_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  certificate_no VARCHAR(80) NOT NULL UNIQUE,
  certificate_level VARCHAR(30) NOT NULL,
  valid_from DATETIME NOT NULL,
  valid_until DATETIME NOT NULL,
  CHECK(valid_until>valid_from),
  FOREIGN KEY(caregiver_id) REFERENCES caregiver(id),
  FOREIGN KEY(item_id) REFERENCES service_item(id));
```

护工服务区域

```
CREATE TABLE caregiver_area(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  caregiver_id BIGINT NOT NULL,
  community_id BIGINT NOT NULL,
  UNIQUE(caregiver_id,community_id),
  FOREIGN KEY(caregiver_id) REFERENCES caregiver(id),
  FOREIGN KEY(community_id) REFERENCES community(id));
```

照护计划

```
CREATE TABLE care_plan(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  assessment_id BIGINT NOT NULL,
  bundle_id BIGINT NOT NULL,
  caregiver_id BIGINT NOT NULL,
  starts_at DATETIME NOT NULL,
  ends_at DATETIME NOT NULL,
  status ENUM('ACTIVE','PAUSED','EXPIRED','ENDED') NOT NULL DEFAULT 'ACTIVE',
  CHECK(ends_at>starts_at),
  FOREIGN KEY(assessment_id) REFERENCES assessment(id),
  FOREIGN KEY(bundle_id) REFERENCES care_bundle(id),
  FOREIGN KEY(caregiver_id) REFERENCES caregiver(id),
  INDEX idx_plan_assessment(assessment_id,status,starts_at,ends_at));
```

计划明细

```
CREATE TABLE plan_item(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  plan_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  weekly_frequency INT NOT NULL,
  CHECK(weekly_frequency>0),
  UNIQUE(plan_id,item_id),
  FOREIGN KEY(plan_id) REFERENCES care_plan(id),
  FOREIGN KEY(item_id) REFERENCES service_item(id));
```

排班

```
CREATE TABLE shift(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  caregiver_id BIGINT NOT NULL,
  starts_at DATETIME NOT NULL,
```

```
  ends_at DATETIME NOT NULL,
  capacity INT NOT NULL,
  used_count INT NOT NULL DEFAULT 0,
  status ENUM('OPEN','CLOSED') NOT NULL DEFAULT 'OPEN',
  CHECK(ends_at>starts_at),
  CHECK(capacity>0 AND used_count>=0 AND used_count<=capacity),
  FOREIGN KEY(caregiver_id) REFERENCES caregiver(id),
  INDEX idx_shift_caregiver(caregiver_id,starts_at,ends_at));
```

服务订单

```
CREATE TABLE service_order(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  elder_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  community_id BIGINT NOT NULL,
  starts_at DATETIME NOT NULL,
  ends_at DATETIME NOT NULL,
  address VARCHAR(200) NOT NULL,
  channel ENUM('PHONE','FAMILY','COMMUNITY') NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  status
ENUM('PENDING','ASSIGNED','ACCEPTED','SERVING','CONFIRM','COMPLETED','CANC
ELLED') NOT NULL DEFAULT 'PENDING',
  created_by BIGINT NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK(ends_at>starts_at),
  CHECK(amount>=0),
  FOREIGN KEY(elder_id) REFERENCES elder(id),
  FOREIGN KEY(item_id) REFERENCES service_item(id),
```

```
  FOREIGN KEY(community_id) REFERENCES community(id),
  FOREIGN KEY(created_by) REFERENCES account(id),
  INDEX idx_order_status_time(status,starts_at),
  INDEX idx_order_elder(elder_id,starts_at));
```

派单记录

```
CREATE TABLE dispatch(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  order_id BIGINT NOT NULL,
  shift_id BIGINT NOT NULL,
  status ENUM('WAITING','ACCEPTED','FINISHED','REASSIGNED','REJECTED') NOT NULL DEFAULT 'WAITING',
  reason VARCHAR(200) NOT NULL DEFAULT '首次派单',
  assigned_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  accepted_at DATETIME,
  active_order BIGINT GENERATED ALWAYS AS (CASE WHEN status IN ('WAITING','ACCEPTED') THEN order_id ELSE NULL END) STORED,
  UNIQUE(active_order),
  FOREIGN KEY(order_id) REFERENCES service_order(id),
  FOREIGN KEY(shift_id) REFERENCES shift(id),
  INDEX idx_dispatch_shift(shift_id,status));
```

服务与确认

```
CREATE TABLE service_record(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  dispatch_id BIGINT NOT NULL UNIQUE,
  notes VARCHAR(1000) NOT NULL,
  confirmed_by BIGINT,
  confirmed_at DATETIME,
  confirmation VARCHAR(300),
```

```
  FOREIGN KEY(dispatch_id) REFERENCES dispatch(id),
  FOREIGN KEY(confirmed_by) REFERENCES account(id));
```

工时记录

```
CREATE TABLE attendance(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  dispatch_id BIGINT NOT NULL UNIQUE,
  check_in DATETIME(6) NOT NULL,
  check_out DATETIME(6),
  location_note VARCHAR(200) NOT NULL,
  fence_ok BOOLEAN NOT NULL,
  status ENUM('PENDING','REVIEW','VALID','INVALID') NOT NULL DEFAULT 'PENDING',
  effective_minutes INT NOT NULL DEFAULT 0,
  reviewer_id BIGINT,
  review_reason VARCHAR(300),
  CHECK(check_out IS NULL OR check_out>check_in),
  CHECK(effective_minutes>=0),
  FOREIGN KEY(dispatch_id) REFERENCES dispatch(id),
  FOREIGN KEY(reviewer_id) REFERENCES account(id));
```

设备

```
CREATE TABLE device(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  serial_no VARCHAR(80) NOT NULL UNIQUE,
  type_name VARCHAR(50) NOT NULL,
  location_note VARCHAR(200) NOT NULL,
  status ENUM('ACTIVE','OFFLINE','RETIRED') NOT NULL DEFAULT 'ACTIVE');
```

绑定历史

```
CREATE TABLE device_binding(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  device_id BIGINT NOT NULL,
```

```
  elder_id BIGINT NOT NULL,
  bound_at DATETIME(6) NOT NULL,
  unbound_at DATETIME(6),
  active_device BIGINT GENERATED ALWAYS AS (CASE WHEN unbound_at IS NULL
THEN device_id ELSE NULL END) STORED,
  UNIQUE(active_device),
  CHECK(unbound_at IS NULL OR unbound_at>bound_at),
  FOREIGN KEY(device_id) REFERENCES device(id),
  FOREIGN KEY(elder_id) REFERENCES elder(id));
```

指标规则

```
CREATE TABLE metric_rule(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  metric VARCHAR(60) NOT NULL UNIQUE,
  low_value DECIMAL(12,3) NOT NULL,
  high_value DECIMAL(12,3) NOT NULL,
  response_seconds INT NOT NULL,
  CHECK(high_value>low_value),
  CHECK(response_seconds>0));
```

体征数据

```
CREATE TABLE vital(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  binding_id BIGINT NOT NULL,
  metric_rule_id BIGINT NOT NULL,
  sample_key VARCHAR(80) NOT NULL,
  collected_at DATETIME NOT NULL,
  metric_value DECIMAL(12,3) NOT NULL,
  abnormal BOOLEAN NOT NULL DEFAULT FALSE,
  source VARCHAR(50) NOT NULL,
```

```
  UNIQUE(binding_id,sample_key),
  FOREIGN KEY(binding_id) REFERENCES device_binding(id),
  FOREIGN KEY(metric_rule_id) REFERENCES metric_rule(id));
```

应急事件

```
CREATE TABLE emergency(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  elder_id BIGINT NOT NULL,
  vital_id BIGINT UNIQUE,
  source VARCHAR(50) NOT NULL,
  level INT NOT NULL DEFAULT 1,
  occurred_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  deadline DATETIME NOT NULL,
  status ENUM('UNCONFIRMED','OPEN','RESPONDED','CLOSED','FALSE_ALARM') NOT
NULL,
  responded_by BIGINT,
  responded_at DATETIME,
  arrived_at DATETIME,
  result VARCHAR(1000),
  CHECK(level BETWEEN 1 AND 3),
  FOREIGN KEY(elder_id) REFERENCES elder(id),
  FOREIGN KEY(vital_id) REFERENCES vital(id),
  FOREIGN KEY(responded_by) REFERENCES account(id),
  INDEX idx_emergency_deadline(status,deadline));
```

升级记录

```
CREATE TABLE emergency_log(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  event_id BIGINT NOT NULL,
  level INT NOT NULL,
```

```
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  message VARCHAR(300) NOT NULL,
  UNIQUE(event_id,level),
  FOREIGN KEY(event_id) REFERENCES emergency(id));
```

通知

```
CREATE TABLE notification(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  event_id BIGINT NOT NULL,
  account_id BIGINT NOT NULL,
  message VARCHAR(300) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(event_id,account_id,message),
  FOREIGN KEY(event_id) REFERENCES emergency(id),
  FOREIGN KEY(account_id) REFERENCES account(id));
```

补贴政策

```
CREATE TABLE policy(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  community_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  ability_level ENUM('SELF','PARTIAL','SEVERE') NOT NULL,
  rate DECIMAL(10,2) NOT NULL,
  starts_at DATETIME NOT NULL,
  ends_at DATETIME NOT NULL,
  CHECK(rate>=0),
  CHECK(ends_at>starts_at),
  FOREIGN KEY(community_id) REFERENCES community(id),
  FOREIGN KEY(item_id) REFERENCES service_item(id));
```

补贴申报

```
CREATE TABLE claim(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  elder_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  period CHAR(7) NOT NULL,
  amount DECIMAL(12,2) NOT NULL DEFAULT 0,
  status
ENUM('DRAFT','SUBMITTED','STREET_APPROVED','APPROVED','REJECTED','PAID')
NOT NULL DEFAULT 'DRAFT',
  UNIQUE(elder_id,item_id,period),
  CHECK(amount>=0),
  FOREIGN KEY(elder_id) REFERENCES elder(id),
  FOREIGN KEY(item_id) REFERENCES service_item(id));
```

申报明细

```
CREATE TABLE claim_item(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  claim_id BIGINT NOT NULL,
  policy_id BIGINT NOT NULL,
  order_id BIGINT NOT NULL UNIQUE,
  amount DECIMAL(12,2) NOT NULL,
  effective_minutes INT NOT NULL,
  rate_snapshot DECIMAL(10,2) NOT NULL,
  CHECK(amount>=0 AND effective_minutes>0),
  FOREIGN KEY(policy_id) REFERENCES policy(id),
  FOREIGN KEY(claim_id) REFERENCES claim(id),
  FOREIGN KEY(order_id) REFERENCES service_order(id));
```

审核记录

```
CREATE TABLE claim_review(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  claim_id BIGINT NOT NULL,
  reviewer_id BIGINT NOT NULL,
  stage ENUM('STREET','REGULATOR') NOT NULL,
  decision ENUM('APPROVE','REJECT') NOT NULL,
  reason VARCHAR(300) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY(claim_id) REFERENCES claim(id),
  FOREIGN KEY(reviewer_id) REFERENCES account(id));
```

拨付

```
CREATE TABLE payment(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  claim_id BIGINT NOT NULL UNIQUE,
  reference_no VARCHAR(80) NOT NULL UNIQUE,
  amount DECIMAL(12,2) NOT NULL,
  paid_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK(amount>0),
  FOREIGN KEY(claim_id) REFERENCES claim(id));
```

质量抽查

```
CREATE TABLE inspection(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  order_id BIGINT,
  caregiver_id BIGINT,
  inspector_id BIGINT NOT NULL,
  method ENUM('PHONE','VISIT') NOT NULL,
  score INT NOT NULL,
  issue VARCHAR(300) NOT NULL,
```

```
  rectification VARCHAR(300) NOT NULL,
  status ENUM('OPEN','CLOSED') NOT NULL DEFAULT 'OPEN',
  result VARCHAR(300),
  CHECK((order_id IS NULL)<>(caregiver_id IS NULL)),
  CHECK(score BETWEEN 0 AND 100),
  FOREIGN KEY(order_id) REFERENCES service_order(id),
  FOREIGN KEY(caregiver_id) REFERENCES caregiver(id),
  FOREIGN KEY(inspector_id) REFERENCES account(id));
```

评价投诉

```
CREATE TABLE complaint(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  order_id BIGINT NOT NULL,
  account_id BIGINT NOT NULL,
  rating INT NOT NULL,
  content VARCHAR(500) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK(rating BETWEEN 1 AND 5),
  FOREIGN KEY(order_id) REFERENCES service_order(id),
  FOREIGN KEY(account_id) REFERENCES account(id));
```

操作审计

```
CREATE TABLE audit_log(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  table_name VARCHAR(50) NOT NULL,
  row_id BIGINT NOT NULL,
  operation VARCHAR(20) NOT NULL,
  actor VARCHAR(100) NOT NULL,
  old_status VARCHAR(100),
  new_status VARCHAR(100),
```

```
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_audit_object(table_name,row_id));
```

```
ALTER TABLE community ADD street_name VARCHAR(80) NOT NULL DEFAULT '演示街道', ADD center_x DECIMAL(12,3) NOT NULL DEFAULT 0, ADD center_y DECIMAL(12,3) NOT NULL DEFAULT 0, ADD radius_m INT NOT NULL DEFAULT 5000, ADD fence_m INT NOT NULL DEFAULT 200, ADD CHECK(radius_m>0 AND fence_m>0 AND fence_m<=radius_m);

ALTER TABLE elder ADD position_x DECIMAL(12,3) NOT NULL DEFAULT 0, ADD position_y DECIMAL(12,3) NOT NULL DEFAULT 0;

ALTER TABLE guardian ADD active BOOLEAN NOT NULL DEFAULT TRUE;

ALTER TABLE service_order ADD position_x DECIMAL(12,3) NOT NULL DEFAULT 0, ADD position_y DECIMAL(12,3) NOT NULL DEFAULT 0;

ALTER TABLE attendance ADD checkin_x DECIMAL(12,3), ADD checkin_y DECIMAL(12,3), ADD checkout_x DECIMAL(12,3), ADD checkout_y DECIMAL(12,3), ADD checkout_fence_ok BOOLEAN, ADD CHECK((checkin_x IS NULL)=(checkin_y IS NULL)), ADD CHECK((checkout_x IS NULL)=(checkout_y IS NULL));

ALTER TABLE device ADD community_id BIGINT, ADD FOREIGN KEY(community_id) REFERENCES community(id);

ALTER TABLE emergency ADD initial_deadline DATETIME, ADD closed_at DATETIME(6), ADD close_reason VARCHAR(300);

ALTER TABLE notification MODIFY event_id BIGINT NULL, ADD assessment_id BIGINT, ADD dedupe_key VARCHAR(150) UNIQUE, ADD FOREIGN KEY(assessment_id) REFERENCES assessment(id);

ALTER TABLE policy ADD settlement_cycle ENUM('MONTH','QUARTER','YEAR') NOT NULL DEFAULT 'MONTH', ADD minimum_age INT NOT NULL DEFAULT 0, ADD required_tag ENUM('ALONE','DISABLED','LOW_INCOME'), ADD CHECK(minimum_age BETWEEN 0 AND 120);

ALTER TABLE claim MODIFY period VARCHAR(10) NOT NULL, ADD community_id BIGINT, ADD period_start DATE, ADD period_end DATE, ADD FOREIGN KEY(community_id) REFERENCES community(id), ADD CHECK(period_end IS NULL OR period_end>period_start);
```

```
ALTER TABLE complaint ADD status
ENUM('SUBMITTED','ASSIGNED','REPLIED','CLOSED') NOT NULL DEFAULT
'SUBMITTED', ADD organization_id BIGINT, ADD handler_id BIGINT, ADD result
VARCHAR(1000), ADD updated_at DATETIME NOT NULL DEFAULT
CURRENT_TIMESTAMP, ADD FOREIGN KEY(organization_id) REFERENCES
organization(id), ADD FOREIGN KEY(handler_id) REFERENCES account(id);

ALTER TABLE audit_log ADD before_data JSON, ADD after_data JSON;
```

监管辖区授权

```
CREATE TABLE account_scope(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  account_id BIGINT NOT NULL,
  community_id BIGINT NOT NULL,
  UNIQUE(account_id,community_id),
  FOREIGN KEY(account_id) REFERENCES account(id),
  FOREIGN KEY(community_id) REFERENCES community(id));
```

政策适用社区

```
CREATE TABLE policy_area(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  policy_id BIGINT NOT NULL,
  community_id BIGINT NOT NULL,
  UNIQUE(policy_id,community_id),
  FOREIGN KEY(policy_id) REFERENCES policy(id),
  FOREIGN KEY(community_id) REFERENCES community(id));
```

招聘申请

```
CREATE TABLE recruitment(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  organization_id BIGINT NOT NULL,
  candidate_name VARCHAR(60) NOT NULL,
  phone_cipher TEXT NOT NULL,
  status ENUM('APPLIED','HIRED','REJECTED') NOT NULL DEFAULT 'APPLIED',
```

```
  caregiver_id BIGINT UNIQUE,
  reviewer_id BIGINT,
  review_note VARCHAR(500),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  reviewed_at DATETIME,
  FOREIGN KEY(organization_id) REFERENCES organization(id),
  FOREIGN KEY(caregiver_id) REFERENCES caregiver(id),
  FOREIGN KEY(reviewer_id) REFERENCES account(id));
```

岗前培训

```
CREATE TABLE training(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  organization_id BIGINT NOT NULL,
  title VARCHAR(100) NOT NULL,
  starts_at DATETIME NOT NULL,
  ends_at DATETIME NOT NULL,
  CHECK(ends_at>starts_at),
  FOREIGN KEY(organization_id) REFERENCES organization(id));
```

培训成绩

```
CREATE TABLE training_record(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  training_id BIGINT NOT NULL,
  caregiver_id BIGINT NOT NULL,
  result ENUM('PENDING','PASS','FAIL') NOT NULL DEFAULT 'PENDING',
  score INT,
  recorded_by BIGINT,
  recorded_at DATETIME,
  UNIQUE(training_id,caregiver_id),
  CHECK(score IS NULL OR score BETWEEN 0 AND 100),
```

```
  FOREIGN KEY(training_id) REFERENCES training(id),
  FOREIGN KEY(caregiver_id) REFERENCES caregiver(id),
  FOREIGN KEY(recorded_by) REFERENCES account(id));
```

投诉办理记录

```
CREATE TABLE complaint_log(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  complaint_id BIGINT NOT NULL,
  actor_id BIGINT NOT NULL,
  status VARCHAR(30) NOT NULL,
  message VARCHAR(1000) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY(complaint_id) REFERENCES complaint(id),
  FOREIGN KEY(actor_id) REFERENCES account(id));
```

比例抽查批次

```
CREATE TABLE inspection_batch(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  community_id BIGINT NOT NULL,
  inspector_id BIGINT NOT NULL,
  target_type ENUM('ORDER','CAREGIVER') NOT NULL,
  starts_at DATETIME NOT NULL,
  ends_at DATETIME NOT NULL,
  ratio DECIMAL(6,5) NOT NULL,
  population INT NOT NULL,
  sample_count INT NOT NULL,
  sample_seed CHAR(36) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK(ends_at>starts_at),
  CHECK(ratio>0 AND ratio<=1),
```

```
  CHECK(population>=sample_count AND sample_count>0),
  FOREIGN KEY(community_id) REFERENCES community(id),
  FOREIGN KEY(inspector_id) REFERENCES account(id));
ALTER TABLE inspection MODIFY status ENUM('PENDING','OPEN','REVIEW','CLOSED')
NOT NULL DEFAULT 'OPEN', ADD batch_id BIGINT, ADD created_at DATETIME NOT
NULL DEFAULT CURRENT_TIMESTAMP, MODIFY score INT NULL, ADD FOREIGN
KEY(batch_id) REFERENCES inspection_batch(id);
```

整改提交与复核

```
CREATE TABLE rectification_submission(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  inspection_id BIGINT NOT NULL,
  submitted_by BIGINT NOT NULL,
  measures VARCHAR(1000) NOT NULL,
  evidence VARCHAR(1000) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  decision ENUM('PENDING','ACCEPT','RETURN') NOT NULL DEFAULT 'PENDING',
  reviewer_id BIGINT,
  review_note VARCHAR(1000),
  reviewed_at DATETIME,
  FOREIGN KEY(inspection_id) REFERENCES inspection(id),
  FOREIGN KEY(submitted_by) REFERENCES account(id),
  FOREIGN KEY(reviewer_id) REFERENCES account(id));
```

机构考核材料

```
CREATE TABLE organization_report(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  organization_id BIGINT NOT NULL,
  period CHAR(7) NOT NULL,
  summary VARCHAR(2000) NOT NULL,
  evidence VARCHAR(1000) NOT NULL,
```

```
  submitted_by BIGINT NOT NULL,
  status ENUM('SUBMITTED','ACCEPTED','RETURNED') NOT NULL DEFAULT
'SUBMITTED',
  reviewer_id BIGINT,
  review_note VARCHAR(1000),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  reviewed_at DATETIME,
  FOREIGN KEY(organization_id) REFERENCES organization(id),
  FOREIGN KEY(submitted_by) REFERENCES account(id),
  FOREIGN KEY(reviewer_id) REFERENCES account(id));
```

服务质量公示

```
CREATE TABLE public_notice(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  inspection_id BIGINT NOT NULL,
  publisher_id BIGINT NOT NULL,
  title VARCHAR(100) NOT NULL,
  body VARCHAR(1000) NOT NULL,
  published_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  withdrawn_at DATETIME,
  withdraw_reason VARCHAR(300),
  FOREIGN KEY(inspection_id) REFERENCES inspection(id),
  FOREIGN KEY(publisher_id) REFERENCES account(id));
```

补贴证明材料

```
CREATE TABLE claim_evidence(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  claim_id BIGINT NOT NULL,
  submitted_by BIGINT NOT NULL,
  name VARCHAR(100) NOT NULL,
```

```
  content TEXT NOT NULL,
  content_sha256 CHAR(64) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY(claim_id) REFERENCES claim(id),
  FOREIGN KEY(submitted_by) REFERENCES account(id));
```

应急处置步骤

```
CREATE TABLE event_step(id BIGINT PRIMARY KEY AUTO_INCREMENT,
  event_id BIGINT NOT NULL,
  actor_id BIGINT NOT NULL,
  action VARCHAR(30) NOT NULL,
  message VARCHAR(1000) NOT NULL,
  created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  FOREIGN KEY(event_id) REFERENCES emergency(id),
  FOREIGN KEY(actor_id) REFERENCES account(id));
CREATE INDEX idx_complaint_status ON complaint(organization_id,status,created_at);
CREATE INDEX idx_scope_community ON account_scope(community_id,account_id);
CREATE INDEX idx_training_worker ON training_record(caregiver_id,result);
CREATE INDEX idx_inspection_batch ON inspection(batch_id,status);
ALTER TABLE complaint ADD kind ENUM('REVIEW','COMPLAINT') NOT NULL DEFAULT
'COMPLAINT';
ALTER TABLE device ADD simulation_enabled BOOLEAN NOT NULL DEFAULT FALSE,
ADD simulation_metric_id BIGINT, ADD simulation_seconds INT NOT NULL DEFAULT 60,
ADD last_simulated_at DATETIME(6), ADD CHECK(simulation_seconds BETWEEN 30
AND 3600), ADD FOREIGN KEY(simulation_metric_id) REFERENCES metric_rule(id);
ALTER TABLE vital MODIFY collected_at DATETIME(6) NOT NULL;
```

业务视图与内部关联视图

五条业务线各提供三个不同用途的视图，共 15 个业务视图。另有 v_plan_context 和 v_dispatch_context 两个内部视图，派生老人、护工归属，避免基础表重复保存。视图提供查询结构，具体人员范围仍由业务授权过滤决定。

```
CREATE VIEW v_assessment_due AS SELECT e.id,e.name,e.community_id,a.id assessment_id,a.valid_until FROM elder e JOIN assessment a ON a.elder_id=e.id WHERE e.status='ACTIVE' AND a.assessed_at<=NOW() AND a.valid_until BETWEEN NOW() AND NOW()+INTERVAL 30 DAY AND NOT EXISTS(SELECT 1 FROM assessment newer WHERE newer.elder_id=a.elder_id AND newer.assessed_at<=NOW() AND (newer.assessed_at>a.assessed_at OR (newer.assessed_at=a.assessed_at AND newer.id>a.id)));

CREATE VIEW v_active_plan AS SELECT p.id,p.elder_id,e.name,e.community_id,p.starts_at,p.ends_at,c.name caregiver_name FROM v_plan_context p JOIN elder e ON e.id=p.elder_id JOIN caregiver c ON c.id=p.caregiver_id WHERE p.status='ACTIVE' AND NOW()>=p.starts_at AND NOW()<p.ends_at;

CREATE VIEW v_family_care AS SELECT g.account_id,e.id elder_id,e.name,e.community_id,p.id plan_id,p.status FROM guardian g JOIN elder e ON e.id=g.elder_id LEFT JOIN v_plan_context p ON p.elder_id=e.id AND p.status='ACTIVE' AND p.starts_at<=NOW() AND p.ends_at>NOW() WHERE g.active=1;

CREATE VIEW v_dispatch_overdue AS SELECT o.id,o.elder_id,o.community_id,o.starts_at,o.status FROM service_order o WHERE (o.status='PENDING' AND o.created_at<NOW()-INTERVAL 15 MINUTE) OR (o.status='ASSIGNED' AND EXISTS(SELECT 1 FROM dispatch d WHERE d.order_id=o.id AND d.status='WAITING' AND d.assigned_at<NOW()-INTERVAL 15 MINUTE));

CREATE VIEW v_wait_confirmation AS SELECT o.id,o.elder_id,o.community_id,r.id record_id FROM service_order o JOIN dispatch d ON d.order_id=o.id JOIN service_record r ON r.dispatch_id=d.id WHERE o.status='CONFIRM' AND r.confirmed_at IS NULL;

CREATE VIEW v_reassign_history AS SELECT d.id,d.order_id,d.caregiver_id,c.name,d.reason,d.assigned_at,o.community_id FROM
```

```
v_dispatch_context d JOIN caregiver c ON c.id=d.caregiver_id JOIN service_order o ON o.id=d.order_id WHERE d.status='REASSIGNED';

CREATE VIEW v_credential_due AS SELECT x.id,c.name,c.organization_id,x.item_id,x.valid_until FROM credential x JOIN caregiver c ON c.id=x.caregiver_id WHERE x.valid_until<NOW()+INTERVAL 30 DAY;

CREATE VIEW v_shift_capacity AS SELECT s.id,s.caregiver_id,c.organization_id,s.starts_at,s.ends_at,s.capacity-s.used_count available FROM shift s JOIN caregiver c ON c.id=s.caregiver_id WHERE s.status='OPEN';

CREATE VIEW v_hours_review AS SELECT a.id,d.caregiver_id,c.organization_id,d.order_id,a.check_in,a.check_out,a.location_note,a.status FROM attendance a JOIN v_dispatch_context d ON d.id=a.dispatch_id JOIN caregiver c ON c.id=d.caregiver_id WHERE a.status='REVIEW';

CREATE VIEW v_device_current AS SELECT d.id,d.serial_no,d.status,b.elder_id,e.community_id,b.bound_at FROM device d LEFT JOIN device_binding b ON b.device_id=d.id AND b.unbound_at IS NULL LEFT JOIN elder e ON e.id=b.elder_id;

CREATE VIEW v_abnormal_vital AS SELECT v.id,b.elder_id,e.community_id,m.metric,v.metric_value,v.collected_at FROM vital v JOIN device_binding b ON b.id=v.binding_id JOIN elder e ON e.id=b.elder_id JOIN metric_rule m ON m.id=v.metric_rule_id WHERE v.abnormal=1;

CREATE VIEW v_emergency_pending AS SELECT x.id,x.elder_id,e.community_id,x.status,x.level,x.deadline,TIMESTAMPDIFF(SECOND,NOW(),x.deadline) remaining_seconds FROM emergency x JOIN elder e ON e.id=x.elder_id WHERE x.status NOT IN('CLOSED','FALSE_ALARM');

CREATE VIEW v_claim_eligible AS SELECT o.id,o.elder_id,o.item_id,o.community_id,o.amount,a.effective_minutes FROM service_order o JOIN dispatch d ON d.order_id=o.id JOIN attendance a ON a.dispatch_id=d.id JOIN service_record r ON r.dispatch_id=d.id JOIN service_item s ON s.id=o.item_id WHERE o.status='COMPLETED' AND d.status='FINISHED' AND r.confirmed_at IS NOT NULL AND a.status='VALID' AND s.subsidizable=1 AND NOT EXISTS(SELECT 1 FROM claim_item ci WHERE ci.order_id=o.id);
```

```
CREATE VIEW v_claim_progress AS SELECT
c.id,c.elder_id,c.community_id,c.period,c.amount,c.status,p.reference_no,p.paid_at FROM
claim c JOIN elder e ON e.id=c.elder_id LEFT JOIN payment p ON p.claim_id=c.id;

CREATE VIEW v_rectification AS SELECT
i.id,i.order_id,i.caregiver_id,i.issue,i.rectification,COALESCE(o.community_id,org.community
_id) community_id FROM inspection i LEFT JOIN service_order o ON o.id=i.order_id LEFT
JOIN caregiver c ON c.id=i.caregiver_id LEFT JOIN organization org ON
org.id=c.organization_id WHERE i.status IN('OPEN','REVIEW');

CREATE VIEW v_plan_context AS SELECT p.*,a.elder_id FROM care_plan p JOIN
assessment a ON a.id=p.assessment_id;

CREATE VIEW v_dispatch_context AS SELECT d.*,s.caregiver_id FROM dispatch d JOIN
shift s ON s.id=d.shift_id;
```

业务查询及两种实现方式

以下 15 组查询均已在同一测试数据集上执行，两种实现的结果逐行比较一致。各组的查询结果和 EXPLAIN 输出保存在 evidence/query-comparison.json。

A1 有效评估与计划的重度失能老人

连接去重与 EXISTS 半连接比较，避免一人多记录重复输出。

实现 A

```
SELECT DISTINCT e.id,e.name FROM elder e JOIN assessment a ON a.elder_id=e.id JOIN
care_plan p ON p.assessment_id=a.id WHERE a.ability_level='SEVERE' AND
p.status='ACTIVE' AND p.starts_at<=NOW() AND p.ends_at>NOW() AND
a.valid_until>NOW();
```

实现 B

```
SELECT e.id,e.name FROM elder e WHERE EXISTS(SELECT 1 FROM assessment a JOIN
care_plan p ON p.assessment_id=a.id WHERE a.elder_id=e.id AND
a.ability_level='SEVERE' AND p.status='ACTIVE' AND p.starts_at<=NOW() AND
p.ends_at>NOW() AND a.valid_until>NOW());
```

A2 每个老人的评估次数

左连接聚合与相关子查询；保留零次评估老人。

实现 A

```
SELECT e.id,COUNT(a.id) n FROM elder e LEFT JOIN assessment a ON a.elder_id=e.id
GROUP BY e.id;
```

实现 B

```
SELECT e.id,(SELECT COUNT(*) FROM assessment a WHERE a.elder_id=e.id) n FROM
elder e;
```

A3 没有生效计划的老人

反连接与 NOT EXISTS，不使用含 NULL 风险的 NOT IN。

实现 A

```
SELECT e.id FROM elder e LEFT JOIN v_plan_context p ON p.elder_id=e.id AND
p.status='ACTIVE' AND p.starts_at<=NOW() AND p.ends_at>NOW() WHERE p.id IS NULL;
```

实现 B

```
SELECT e.id FROM elder e WHERE NOT EXISTS(SELECT 1 FROM v_plan_context p
WHERE p.elder_id=e.id AND p.status='ACTIVE' AND p.starts_at<=NOW() AND
p.ends_at>NOW());
```

B1 有改派历史的订单

多表连接去重与 EXISTS，结果粒度为订单。

实现 A

```
SELECT DISTINCT o.id,e.name FROM service_order o JOIN elder e ON e.id=o.elder_id
JOIN dispatch d ON d.order_id=o.id WHERE d.status='REASSIGNED';
```

实现 B

```
SELECT o.id,e.name FROM service_order o JOIN elder e ON e.id=o.elder_id WHERE EXISTS(SELECT 1 FROM v_dispatch_context d WHERE d.order_id=o.id AND d.status='REASSIGNED');
```

B2 各社区订单金额

聚合与相关子查询，空集合显式转零。

实现 A

```
SELECT c.id,COALESCE(SUM(o.amount),0) total FROM community c LEFT JOIN service_order o ON o.community_id=c.id GROUP BY c.id;
```

实现 B

```
SELECT c.id,COALESCE((SELECT SUM(o.amount) FROM service_order o WHERE o.community_id=c.id),0) total FROM community c;
```

B3 待确认订单

保留订单粒度，防止多次派单重复统计。

实现 A

```
SELECT DISTINCT o.id FROM service_order o JOIN dispatch d ON d.order_id=o.id JOIN service_record r ON r.dispatch_id=d.id WHERE o.status='CONFIRM' AND r.confirmed_at IS NULL;
```

实现 B

```
SELECT o.id FROM service_order o WHERE o.status='CONFIRM' AND EXISTS(SELECT 1 FROM v_dispatch_context d JOIN service_record r ON r.dispatch_id=d.id WHERE d.order_id=o.id AND r.confirmed_at IS NULL);
```

C1 护工有效工时

三表聚合与子查询。未服务护工仍返回零。

实现 A

SELECT c.id,COALESCE(SUM(a.effective_minutes),0) minutes FROM caregiver c LEFT JOIN shift sh ON sh.caregiver_id=c.id LEFT JOIN dispatch d ON d.shift_id=sh.id LEFT JOIN attendance a ON a.dispatch_id=d.id AND a.status='VALID' GROUP BY c.id;

实现 B

SELECT c.id,COALESCE((SELECT SUM(a.effective_minutes) FROM shift sh JOIN dispatch d ON d.shift_id=sh.id JOIN attendance a ON a.dispatch_id=d.id WHERE sh.caregiver_id=c.id AND a.status='VALID'),0) minutes FROM caregiver c;

C2 有空余名额的护工

该查询只检查容量，不等同于完整派单资格推荐。

实现 A

SELECT DISTINCT c.id FROM caregiver c JOIN shift s ON s.caregiver_id=c.id WHERE s.status='OPEN' AND s.used_count<s.capacity;

实现 B

SELECT c.id FROM caregiver c WHERE EXISTS(SELECT 1 FROM shift s WHERE s.caregiver_id=c.id AND s.status='OPEN' AND s.used_count<s.capacity);

C3 证书到期护工

到期清单与完整资质判断分离，一张证书到期不代表所有项目失效。

实现 A

SELECT DISTINCT c.id FROM caregiver c JOIN credential x ON x.caregiver_id=c.id WHERE x.valid_until<NOW();

实现 B

SELECT c.id FROM caregiver c WHERE EXISTS(SELECT 1 FROM credential x WHERE x.caregiver_id=c.id AND x.valid_until<NOW());

D1 老人异常体征数量

绑定历史关联，不能仅通过设备当前归属统计。

实现 A

```
SELECT e.id,COUNT(v.id) n FROM elder e LEFT JOIN device_binding b ON b.elder_id=e.id
LEFT JOIN vital v ON v.binding_id=b.id AND v.abnormal=1 GROUP BY e.id;
```

实现 B

```
SELECT e.id,(SELECT COUNT(*) FROM device_binding b JOIN vital v ON v.binding_id=b.id
WHERE b.elder_id=e.id AND v.abnormal=1) n FROM elder e;
```

D2 存在未关闭事件的老人

连接去重与 EXISTS，待确认事件仍纳入。

实现 A

```
SELECT DISTINCT e.id FROM elder e JOIN emergency x ON x.elder_id=e.id WHERE
x.status NOT IN('CLOSED','FALSE_ALARM');
```

实现 B

```
SELECT e.id FROM elder e WHERE EXISTS(SELECT 1 FROM emergency x WHERE
x.elder_id=e.id AND x.status NOT IN('CLOSED','FALSE_ALARM'));
```

D3 当前未绑定设备

历史绑定不应排除当前空闲设备。

实现 A

```
SELECT d.id FROM device d LEFT JOIN device_binding b ON b.device_id=d.id AND
b.unbound_at IS NULL WHERE b.id IS NULL;
```

实现 B

```
SELECT d.id FROM device d WHERE NOT EXISTS(SELECT 1 FROM device_binding b
WHERE b.device_id=d.id AND b.unbound_at IS NULL);
```

E1 按社区汇总已申报金额

按申报时保存的社区汇总；老人迁移后不改变已申报金额的归属。

实现 A

```
SELECT co.id,COALESCE(SUM(c.amount),0) amount FROM community co LEFT JOIN claim c ON c.community_id=co.id AND c.status<>'DRAFT' GROUP BY co.id;
```

实现 B

```
SELECT co.id,COALESCE((SELECT SUM(c.amount) FROM claim c WHERE c.community_id=co.id AND c.status<>'DRAFT'),0) amount FROM community co;
```

E2 待拨付申报

反连接与 NOT EXISTS。

实现 A

```
SELECT c.id FROM claim c LEFT JOIN payment p ON p.claim_id=c.id WHERE c.status='APPROVED' AND p.id IS NULL;
```

实现 B

```
SELECT c.id FROM claim c WHERE c.status='APPROVED' AND NOT EXISTS(SELECT 1 FROM payment p WHERE p.claim_id=c.id);
```

E3 各申报明细合计

汇总后与主表金额比较，用于核算一致性检查。

实现 A

```
SELECT c.id,COALESCE(SUM(i.amount),0) amount FROM claim c LEFT JOIN claim_item i ON i.claim_id=c.id GROUP BY c.id;
```

实现 B

```
SELECT c.id,COALESCE((SELECT SUM(i.amount) FROM claim_item i WHERE i.claim_id=c.id),0) amount FROM claim c;
```

六、项目运维管理和优化

环境和边界

Java 17、Spring Boot 3.4.4、Spring JDBC，真实 MySQL 8.4.11。本机独立实例绑定 127.0.0.1:33316，应用绑定 127.0.0.1:18080。Windows 使用安装好的 Java 和 MySQL，通过配置更换 JDBC 地址，Docker 不参与运行。

【待确认】Windows 脚本需要在目标电脑执行，补充初始化、启动和业务操作结果。

设备上传和地理围栏为课程模拟。指标上下限、设备规则配置的初始响应时限、人工事件按级别采用 300/180/60 秒、升级后按级别采用 180/60 秒及每次 30 元补贴均为演示配置，不能用于真实医疗或财政业务。补贴按政策选择月、季或年结算，同一老人同一项目的申报周期不得重叠。驳回后在原申报补充材料并重新提交，原订单明细与历次审核不删除。

过程与触发器对应

业务线	过程	触发器与作用
档案与评估	expire_plans	plan_before_insert / update 调用 assert_plan 检查评估和计划重叠
下单与派单	assign_order	order_before_insert 检查有效照护，order_before_update 检查确认凭据和完成后不可改
护工与排班	verify_hours	shift_before_insert / update 检查重叠班次；CHECK 保护容量范围
监测与应急	escalate_events	vital_before_insert 校验绑定时刻和阈值，vital_after_insert 建待确认事件

业务线	过程	触发器与作用
补贴与监管	submit_claim	claim_before_update 保护审核流转、明细金额与已拨付记录

过程由 Java 事务调用，过程内部不 COMMIT；一个业务方法失败时统一回滚，不能在调用链中提前提交。定时任务每 30 秒检查到期和超时，属于应用进程内调度，进程停止后暂停，恢复后补处理。重要入口还会即时校验有效期，不能只依赖定时状态更新。

索引方案

1. idx_assessment_elder(elder_id,valid_until)：按老人查有效评估，适用于资格判断和复评清单。纯到期全库扫描可能仍需时间列开头的索引，应根据实际规模再决定。

2. idx_plan_assessment(assessment_id,status,starts_at,ends_at)：关联评估并筛选计划。重叠判断仍需比较结束时刻，不能把索引等同于区间约束。

3. idx_order_status_time(status,starts_at)：调度状态与预约区间查询。证据中对比忽略该索引与使用该索引的 EXPLAIN；当前数据量较小，性能结论需用更大数据集验证。

4. idx_order_elder(elder_id,starts_at)：老人订单分页、历史服务和月度申报筛选。

5. idx_shift_caregiver(caregiver_id,starts_at,ends_at)：查护工时段及候选班次。

6. idx_emergency_deadline(status,deadline)：定时筛选未响应且超时的应急事件。

7. idx_audit_object(table_name,row_id)：追溯指定业务记录的状态历史。

多对多表使用组合 UNIQUE 防止重复关系。生成唯一列 active_order 和 active_device 分别保证一单只有一个有效派单、一设备只有一个当前绑定。外键自动或显式建立必要访问索引，避免对每个字段机械加索引。

登录和敏感字段

密码以 BCrypt 12 轮加盐摘要保存。连续失败 5 次后锁定 15 分钟，失败计数使用独立数据库事务提交，避免抛出登录错误时把计数一起回滚。会话由服务端保存随机令牌，固定 30 分钟到期，退出立即删除，应用重启使既有会话失效。每次请求重新检查账户状态和授权范围，账号停用、角色或归属社区变更后旧令牌失效；多社区授权范围的修改在下一次查询时生效。当前是单实例课程实现，未做分布式会话共享。

身份字段以 AES GCM 加密，用 HMAC SHA-256 支持唯一查重；当前示例共用一份应用字段密钥分别调用加密与 HMAC，生产应实施独立密钥、轮换与外部密钥管理。密钥只保存在本机 .local 配置中，不进入版本库或分发包。档案查询不返回身份密文、查重摘要或密码摘要。

所有用户值通过 JDBC 参数绑定。可选数据类别与存储过程名使用固定白名单，不能将用户传入的表名或 SQL 直接执行。角色权限在业务入口检查，家属关系、护工订单及机构辖区再决定行范围。服务仅绑定本机回环地址，若改为远程访问需增加 HTTPS 和部署侧访问限制。

备份与恢复

【待确认】备份方案暂定为每日全量备份，每 15 分钟复制新增 binlog，保留最近 7 天。目标恢复点为 15 分钟以内，目标恢复时间为 30 分钟。目前尚未设置长期定时任务，也未测量这两个目标，需在实际部署时验证。

全量使用 mysqldump --single-transaction --source-data=2 --routines --triggers --events，保存结构、数据和例程。备份期间避免 DDL，记录与一致性快照对应的日志位置。实际部署应采用 dump 的日志位置信息或受控写入暂停来衔接全量与增量；不能随便把完成导出后

的任意日志位置作为起点。演练脚本在受控课程实例上执行，演示单条事务的日志恢复，不等于覆盖所有生产故障。

恢复步骤：停止目标业务写入；在独立空库导入全量；按开始和结束日志位置重放 binlog；核对各基础表行数、外键和例程，再运行核心查询及业务拒绝测试；确认后再切换业务连接。禁止直接在未知的已有数据库中覆盖恢复。

实际证据见 evidence/recovery.json：全量恢复后重放一条真实的新增服务项目事务，并逐行比较 47 张基础表的内容，再执行恢复库中的 15 组查询对比。日志起点取自全量备份内的一致性快照坐标。备份文件在 .local/backups，防止把含账号摘要和字段密文的数据随源码公开。

已遇到的问题与处理

1. 本机 Homebrew MySQL 的动态链接库缺失，Docker 内部磁盘也已满。最终复用可工作的 MySQL 8.4 程序，以独立数据目录和端口启动原生实例，不改动原实验数据库。

2. 初始表结构重复保存了计划所属老人和派单所属护工。根据函数依赖拆除重复字段，通过两个内部视图还原查询上下文，迁移时保留业务记录，然后重跑数据库集成测试。

3. 完整流程测试暴露了 JDBC 日期类型差异和直接访问被代理 DAO 字段的问题。分别统一时间对象处理、将访问封装回 DAO 方法，增加完整服务确认与工时复核的回归测试。

【待补充】小组讨论、成员协作和教师反馈记录。

本次需求与代码复查

派单、接单和签到共用资格检查。除有效评估外，还检查护工在岗、账号启用、证书覆盖服务时段、区域、班次和整改。推荐结果只供选择，真正派单时仍在事务锁内重新核验。按小时计费时，订单价格按标准分钟数折算。

超时调度池由视图自动列出创建超过 15 分钟的待派订单，以及当前派单超过 15 分钟仍未接单的订单。界面提供查询和人工改派。评估到期提醒、异常工时、可申报订单等 15 个业务视图均接入有权限限制的查询入口。

设备绑定使用微秒时间，禁止改写历史老人归属和重叠绑定区间；解绑不能切断已有采样的有效区间。新应急事件立即产生家属、责任护工和社区的站内通知，超时升级后再次通知，人工上报保留在事件日志中。未接入短信或电话网关。

工时汇总按签到月份统计有效分钟。护工评价分别展示家属的 5 分制评分和抽查的 100 分制评分，机构评价汇总所属护工及其已完成订单的抽查结果，不虚构综合评分权重。

监管账号通过 account_scope 关联多个社区；街道和民政分别使用各自授权集合。辖区统计按当前在册老人计算本月服务覆盖率，按非误报且保存首次截止时间的本月事件计算及时响应率。老人归属采用当前社区；已申报和已拨付金额按申报快照中的社区汇总。分母为零时显示“—”，不显示虚假的 100%。迁移前缺少首次时限的旧事件不倒推补填。

新增流程的验收口径

招聘分登记、录用或拒绝；录用建立护工账号和默认服务区域。培训单独安排、登记参训和成绩，结束后才能登记分数；课程配置为 60 分通过。派单同时检查已通过的培训与有效项目证书，两者不能相互替代。

服务评价与投诉分开处理。评价要求订单已完成；投诉依次为提交、社区受理、机构或社区回复和家属或社区结案。办理记录只追加，每一步保存经办账号和说明。家属能够查看订单的服务内容、实际护工、工时与确认凭据。

社区服务范围和围栏采用虚构平面坐标，单位为米；住址不能超出所属社区半径，订单保存下单时的坐标。签到和签退均由服务端计算距离，不接收客户端直接指定“通过”。围

栏或预约时段异常、实际服务不足预约时长 80% 的工时进入复核；核准零分钟服务被拒绝。80%、前后 30 分钟及围栏半径均为课程演示规则，不是任务书给定的行业标准。

每台设备由管理员明确开启定时模拟采样，间隔为 30—3600 秒，默认关闭。任务每 30 秒检查到期设备，在当前绑定下生成正常范围模拟值，记录上次采样时间；异常值由批量上传入口演示。设备离线、停用或解绑时不生成定时样本。通知为站内记录，不接入真实短信或硬件。

监管人员按社区、服务期间及对象类别建立抽查批次。系统对符合条件的已完成订单或其护工排序抽样，样本数为总体乘比例后向上取整。批次保留范围、总体数、种子及实际抽中记录；历史抽中记录不变，之后新增的服务记录不会自动加入旧批次。低于 60 分或明确要求整改的检查进入整改流程。机构可提交材料，监管可通过或退回；退回后保留旧材料并追加新提交。公示仅包含方式、评分和完成状态，隐去身份、地址和健康、投诉明细。

版本升级

2026-11-13 通过 20261113_completion.sql 从原 35 表版本升级到 47 表。执行前先停应用并保存私有全量备份，先在 eldercare_migration 演练再更新 eldercare。原 35 表已有行的原始列逐行散列一致；新增字段回填与新增虚构样例单独记录。升级脚本不能重复导入，也不能用于已经执行最新 01、02、03 脚本的空库初始化。

真实 MySQL 测试总数、失败数及每个方法名见 evidence/test-summary.json。浏览器验证记录位于 evidence/completion-browser.json；数据库迁移校验位于 evidence/migration-eldercare.json。报告仅将已通过的结果写为完成，真实团队材料与 Windows 实机运行仍由蓝色文字标注。

复评 迁移与通知的衔接规则

这些是依据任务书业务关系确定的项目实现规则，任务书没有逐条规定下列处置方式。复评采用已经生效的最新一条评估，时间相同时以较大的评估编号为准。登记当前复评后，旧评估对应的生效计划自动暂停；先处理未完成订单，再按新评估调整计划。旧评估不能重新用于生效计划，下单、派单和接单会再次检查。补录更早的评估不影响当前计划；未来评估到时生效后由定时任务暂停旧计划。已完成服务、原有工时和已拨付补贴不改写，补贴资格仍按服务发生时的评估判断。

老人跨社区迁移前，必须处理未完成订单并解绑原社区设备。迁移保留老人编号，暂停原计划；原设备仍归原社区管理。迁入社区可以为老人绑定本社区设备。原设备绑定区间、体征和历史服务保持原有记录，数据库也拒绝绕过页面直接迁移仍绑定设备的老人。

一级事件通知家属、责任护工和本社区管理员；二级及以上增加本社区监管人员。未响应事件超时后逐级升级并追加升级日志。获得其他社区查询授权的监管账号可以查看相应事件，但不会因此全部订阅该社区通知。通知是站内记录，实际值班轮换与短信发送不在本次实现范围。

已完成、已改派和已拒绝的派单不可更新或删除。尚未结束的派单也不能改写原派单时间或已登记的接单时间，不能从已接单退回等待接单。应急升级日志只追加，禁止更新与删除。这些保护在数据库执行，直接使用应用数据库账号也不能绕过。

历史保护规则升级

已有 47 表版本使用 database/migrations/20261113_history_rules.sql；新安装直接执行最新 01、02、03 脚本，不再另行执行旧版本迁移。升级前停止应用并保存包含例程与触发器的全量备份。脚本仅替换 11 个存储对象，可以重复执行，不修改业务数据。开发机已在

副本连续执行两次，再升级展示库；47 表行数及逐行内容散列保持一致，升级后的例程和触发器定义与新建测试库一致。

升级后执行 125 项 MySQL 集成测试，包含 16 项本轮回归用例。15 组查询继续逐行比较一致，空结果行数为 0；全量加 binlog 恢复再次核对 47 张表。具体结果见 evidence/test-summary.json、query-comparison.json、history-migration-eldercare.json 和 recovery.json。

存储过程与触发器代码

以下核心例程按业务覆盖有效期、派单、工时、应急与补贴，全部审计和历史保护触发器在源代码附件中提供。

```
DELIMITER $$
CREATE PROCEDURE assert_plan(IN p_elder BIGINT,IN p_assess BIGINT,IN p_bundle BIGINT,IN p_start DATETIME,IN p_end DATETIME,IN p_id BIGINT)
BEGIN
 DECLARE guard_id BIGINT;
 SELECT id INTO guard_id FROM elder WHERE id=p_elder FOR UPDATE;
 IF NOT EXISTS(SELECT 1 FROM elder WHERE id=p_elder AND status='ACTIVE') THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='ELDER_INACTIVE'; END IF;
 IF NOT EXISTS(SELECT 1 FROM assessment a JOIN care_bundle b ON b.ability_level=a.ability_level WHERE a.id=p_assess AND a.elder_id=p_elder AND b.id=p_bundle AND a.assessed_at<=p_start AND a.valid_until>=p_end) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='PLAN_ASSESSMENT_INVALID'; END IF;
 IF NOT(p_assess<=>(SELECT id FROM assessment WHERE elder_id=p_elder AND assessed_at<=NOW() ORDER BY assessed_at DESC,id DESC LIMIT 1)) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='PLAN_ASSESSMENT_SUPERSEDED'; END IF;
 IF EXISTS(SELECT 1 FROM v_plan_context WHERE elder_id=p_elder AND id<>p_id AND status='ACTIVE' AND starts_at<p_end AND ends_at>p_start) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='PLAN_OVERLAP'; END IF;
END$$
```

```
CREATE TRIGGER plan_before_insert BEFORE INSERT ON care_plan FOR EACH ROW BEGIN IF NEW.status='ACTIVE' THEN CALL assert_plan((SELECT elder_id FROM assessment WHERE id=NEW.assessment_id),NEW.assessment_id,NEW.bundle_id,NEW.starts_at,NEW.ends_at,0); END IF; END$$

CREATE TRIGGER plan_before_update BEFORE UPDATE ON care_plan FOR EACH ROW BEGIN IF NEW.status='ACTIVE' THEN CALL assert_plan((SELECT elder_id FROM assessment WHERE id=NEW.assessment_id),NEW.assessment_id,NEW.bundle_id,NEW.starts_at,NEW.ends_at,NEW.id); END IF; END$$

CREATE PROCEDURE expire_plans()

BEGIN

 UPDATE care_plan p JOIN assessment a ON a.id=p.assessment_id SET p.status=CASE WHEN a.valid_until<=NOW() THEN 'PAUSED' WHEN p.ends_at<=NOW() THEN 'EXPIRED' ELSE 'PAUSED' END WHERE p.status='ACTIVE' AND (a.valid_until<=NOW() OR p.ends_at<=NOW() OR EXISTS(SELECT 1 FROM assessment newer WHERE newer.elder_id=a.elder_id AND newer.assessed_at<=NOW() AND (newer.assessed_at>a.assessed_at OR (newer.assessed_at=a.assessed_at AND newer.id>a.id))));

END$$

CREATE PROCEDURE assert_shift(IN p_worker BIGINT,IN p_start DATETIME,IN p_end DATETIME,IN p_id BIGINT)

BEGIN

 DECLARE guard_id BIGINT;

 SELECT id INTO guard_id FROM caregiver WHERE id=p_worker FOR UPDATE;

 IF EXISTS(SELECT 1 FROM shift WHERE caregiver_id=p_worker AND id<>p_id AND status='OPEN' AND starts_at<p_end AND ends_at>p_start) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='SHIFT_OVERLAP'; END IF;

END$$

CREATE TRIGGER shift_before_insert BEFORE INSERT ON shift FOR EACH ROW BEGIN CALL assert_shift(NEW.caregiver_id,NEW.starts_at,NEW.ends_at,0); END$$
```

```
CREATE TRIGGER shift_before_update BEFORE UPDATE ON shift FOR EACH ROW
BEGIN IF NEW.status='OPEN' THEN CALL
assert_shift(NEW.caregiver_id,NEW.starts_at,NEW.ends_at,NEW.id); END IF; END$$

CREATE PROCEDURE assert_order(IN p_elder BIGINT,IN p_item BIGINT,IN p_community
BIGINT,IN p_start DATETIME,IN p_end DATETIME)

BEGIN

 DECLARE guard_id BIGINT;

 SELECT id INTO guard_id FROM elder WHERE id=p_elder FOR UPDATE;

 IF NOT EXISTS(SELECT 1 FROM elder WHERE id=p_elder AND
community_id=p_community AND status='ACTIVE') THEN SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT='ORDER_COMMUNITY_INVALID'; END IF;

 IF NOT EXISTS(SELECT 1 FROM v_plan_context p JOIN assessment a ON
a.id=p.assessment_id JOIN plan_item i ON i.plan_id=p.id WHERE p.elder_id=p_elder AND
i.item_id=p_item AND p.status='ACTIVE' AND p.starts_at<=p_start AND
p.ends_at>=p_end AND a.valid_until>=p_end AND a.valid_until>NOW() AND a.id=(SELECT
id FROM assessment WHERE elder_id=p_elder AND assessed_at<=NOW() ORDER BY
assessed_at DESC,id DESC LIMIT 1)) THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='ORDER_PLAN_INVALID'; END IF;

END$$

CREATE TRIGGER order_before_insert BEFORE INSERT ON service_order FOR EACH
ROW BEGIN CALL
assert_order(NEW.elder_id,NEW.item_id,NEW.community_id,NEW.starts_at,NEW.ends_at);
END$$

CREATE TRIGGER order_before_update BEFORE UPDATE ON service_order FOR EACH
ROW

BEGIN

 IF OLD.status='COMPLETED' THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='COMPLETED_ORDER_IMMUTABLE'; END IF;

 IF NEW.status='COMPLETED' AND NOT EXISTS(SELECT 1 FROM v_dispatch_context d
JOIN service_record r ON r.dispatch_id=d.id WHERE d.order_id=NEW.id AND
d.status='FINISHED' AND r.confirmed_by IS NOT NULL AND r.confirmed_at IS NOT NULL
```

```
AND r.confirmation IS NOT NULL AND EXISTS(SELECT 1 FROM guardian g JOIN account ac ON ac.id=g.account_id WHERE g.elder_id=NEW.elder_id AND g.account_id=r.confirmed_by AND g.active=1 AND ac.role='FAMILY')) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='CONFIRMATION_REQUIRED'; END IF;

END$$

-- Assignment, acceptance and check-in share the same eligibility checks.

CREATE PROCEDURE assert_worker_order(IN p_order BIGINT,IN p_shift BIGINT)

BEGIN

 DECLARE v_worker,v_guard,v_elder,v_item,v_community BIGINT;

 DECLARE v_start,v_end DATETIME;

 SELECT elder_id,item_id,community_id,starts_at,ends_at INTO v_elder,v_item,v_community,v_start,v_end FROM service_order WHERE id=p_order;

 CALL assert_order(v_elder,v_item,v_community,v_start,v_end);

 SELECT caregiver_id INTO v_worker FROM shift WHERE id=p_shift;

 SELECT id INTO v_guard FROM caregiver WHERE id=v_worker FOR UPDATE;

 IF v_worker IS NULL OR NOT EXISTS(SELECT 1 FROM caregiver c JOIN account ac ON ac.id=c.account_id JOIN caregiver_area a ON a.caregiver_id=c.id JOIN credential x ON x.caregiver_id=c.id WHERE c.id=v_worker AND c.status='ACTIVE' AND ac.active=1 AND a.community_id=v_community AND x.item_id=v_item AND x.valid_from<=LEAST(NOW(),v_start) AND x.valid_until>=GREATEST(NOW(),v_end)) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='WORKER_NOT_QUALIFIED'; END IF;

 IF NOT EXISTS(SELECT 1 FROM training_record WHERE caregiver_id=v_worker AND result='PASS') THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='TRAINING_REQUIRED'; END IF;

 IF NOT EXISTS(SELECT 1 FROM shift WHERE id=p_shift AND status='OPEN' AND starts_at<=v_start AND ends_at>=v_end) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='SHIFT_FULL_OR_OUTSIDE'; END IF;

 IF EXISTS(SELECT 1 FROM inspection i WHERE i.status IN('OPEN','REVIEW') AND (i.caregiver_id=v_worker OR i.order_id IN(SELECT order_id FROM v_dispatch_context
```

```
WHERE caregiver_id=v_worker AND status IN('ACCEPTED','FINISHED')))) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='RECTIFICATION_REQUIRED'; END IF;

 IF EXISTS(SELECT 1 FROM v_dispatch_context d JOIN service_order o ON o.id=d.order_id WHERE d.caregiver_id=v_worker AND d.status IN('WAITING','ACCEPTED') AND o.id<>p_order AND o.starts_at<v_end AND o.ends_at>v_start) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='WORKER_TIME_CONFLICT'; END IF;

END$$

CREATE PROCEDURE assign_order(IN p_order BIGINT,IN p_shift BIGINT,IN p_reason VARCHAR(200))

BEGIN

 DECLARE v_elder,v_item,v_community,v_worker,v_guard,v_old_shift BIGINT;

 DECLARE v_start,v_end DATETIME;

 DECLARE v_state VARCHAR(30);

 SELECT elder_id,item_id,community_id,starts_at,ends_at,status INTO v_elder,v_item,v_community,v_start,v_end,v_state FROM service_order WHERE id=p_order FOR UPDATE;

 IF v_state IS NULL OR v_state NOT IN ('PENDING','ASSIGNED','ACCEPTED') THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='ORDER_NOT_DISPATCHABLE'; END IF;

 CALL assert_worker_order(p_order,p_shift);

 SET v_old_shift=(SELECT shift_id FROM v_dispatch_context WHERE order_id=p_order AND active_order IS NOT NULL);

 IF v_old_shift IS NOT NULL THEN

  IF p_reason IS NULL OR LENGTH(TRIM(p_reason))<2 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='REASSIGN_REASON_REQUIRED'; END IF;

  UPDATE dispatch SET status='REASSIGNED',reason=p_reason WHERE order_id=p_order AND active_order IS NOT NULL;

  UPDATE shift SET used_count=used_count-1 WHERE id=v_old_shift;

 END IF;
```

```
 UPDATE shift SET used_count=used_count+1 WHERE id=p_shift AND status='OPEN'
AND starts_at<=v_start AND ends_at>=v_end AND used_count<capacity;

 IF ROW_COUNT()<>1 THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='SHIFT_FULL_OR_OUTSIDE'; END IF;

 INSERT INTO dispatch(order_id,shift_id,reason)
VALUES(p_order,p_shift,COALESCE(p_reason,'首次派单'));

 UPDATE service_order SET status='ASSIGNED' WHERE id=p_order;

END$$

CREATE PROCEDURE verify_hours(IN p_dispatch BIGINT)

BEGIN

 UPDATE attendance a JOIN v_dispatch_context d ON d.id=a.dispatch_id JOIN
service_order o ON o.id=d.order_id SET a.status=IF(a.fence_ok AND
COALESCE(a.checkout_fence_ok,0) AND a.check_in>=o.starts_at-INTERVAL 30 MINUTE
AND a.check_out<=o.ends_at+INTERVAL 30 MINUTE AND
TIMESTAMPDIFF(MINUTE,a.check_in,a.check_out)>=CEIL(TIMESTAMPDIFF(MINUTE,o.st
arts_at,o.ends_at)*0.8),'VALID','REVIEW'),a.effective_minutes=IF(a.fence_ok AND
COALESCE(a.checkout_fence_ok,0) AND a.check_in>=o.starts_at-INTERVAL 30 MINUTE
AND a.check_out<=o.ends_at+INTERVAL 30 MINUTE AND
TIMESTAMPDIFF(MINUTE,a.check_in,a.check_out)>=CEIL(TIMESTAMPDIFF(MINUTE,o.st
arts_at,o.ends_at)*0.8),TIMESTAMPDIFF(MINUTE,a.check_in,a.check_out),0) WHERE
a.dispatch_id=p_dispatch AND a.check_out IS NOT NULL AND a.status='PENDING';

END$$

CREATE TRIGGER vital_before_insert BEFORE INSERT ON vital FOR EACH ROW

BEGIN

 DECLARE v_low,v_high DECIMAL(12,3);

 IF NEW.collected_at>NOW(6) THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='VITAL_FUTURE_TIME'; END IF;

 IF NOT EXISTS(SELECT 1 FROM device_binding WHERE id=NEW.binding_id AND
bound_at<=NEW.collected_at AND (unbound_at IS NULL OR
unbound_at>NEW.collected_at)) THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='VITAL_BINDING_TIME_INVALID'; END IF;
```

```
 SELECT low_value,high_value INTO v_low,v_high FROM metric_rule WHERE
id=NEW.metric_rule_id;

 SET NEW.abnormal=(NEW.metric_value<v_low OR NEW.metric_value>v_high);

END$$

CREATE TRIGGER vital_after_insert AFTER INSERT ON vital FOR EACH ROW

BEGIN

 IF NEW.abnormal THEN

 INSERT INTO emergency(elder_id,vital_id,source,deadline,status) SELECT
b.elder_id,NEW.id,'DEVICE',DATE_ADD(NOW(),INTERVAL m.response_seconds
SECOND),'UNCONFIRMED' FROM device_binding b JOIN metric_rule m ON
m.id=NEW.metric_rule_id WHERE b.id=NEW.binding_id;

 END IF;

END$$

CREATE PROCEDURE escalate_events()

BEGIN

 INSERT IGNORE INTO emergency_log(event_id,level,message) SELECT id,level+1,'响应超
时，升级并通知值班主管' FROM emergency WHERE status IN('OPEN','UNCONFIRMED')
AND deadline<=NOW() AND level<3;

 UPDATE emergency SET level=level+1,deadline=DATE_ADD(NOW(),INTERVAL
IF(level=2,180,60) SECOND) WHERE status IN('OPEN','UNCONFIRMED') AND
deadline<=NOW() AND level<3;

 INSERT IGNORE INTO notification(event_id,account_id,message)

 SELECT e.id,a.id,CONCAT('应急事件待处理，级别 ',e.level)

 FROM emergency e JOIN elder x ON x.id=e.elder_id JOIN account a ON a.active=1

 WHERE e.status IN('OPEN','UNCONFIRMED') AND (

  (a.community_id=x.community_id AND (a.role='ADMIN' OR (a.role='REGULATOR' AND
e.level>=2)))

  OR EXISTS(SELECT 1 FROM guardian g WHERE g.elder_id=e.elder_id AND
g.account_id=a.id AND g.active=1)
```

```
  OR EXISTS(SELECT 1 FROM v_plan_context p JOIN caregiver c ON c.id=p.caregiver_id
WHERE p.elder_id=e.elder_id AND p.status='ACTIVE' AND p.starts_at<=NOW() AND
p.ends_at>NOW() AND c.account_id=a.id AND c.status='ACTIVE')

  OR EXISTS(SELECT 1 FROM service_order o JOIN v_dispatch_context d ON
d.order_id=o.id JOIN caregiver c ON c.id=d.caregiver_id WHERE o.elder_id=e.elder_id
AND d.status IN('WAITING','ACCEPTED') AND c.account_id=a.id AND c.status='ACTIVE')

 );

END$$

CREATE PROCEDURE submit_claim(IN p_elder BIGINT,IN p_policy BIGINT,IN p_period
VARCHAR(10),IN p_orders JSON)

BEGIN

 DECLARE v_guard,v_item,v_claim,v_order,v_community BIGINT;

 DECLARE v_rate,v_amount,v_price DECIMAL(12,2);

 DECLARE v_minutes,v_i INT DEFAULT 0;

 DECLARE v_unit,v_cycle VARCHAR(10);

 DECLARE v_start,v_end DATE;

 SELECT id,community_id INTO v_guard,v_community FROM elder WHERE id=p_elder
FOR UPDATE;

 SELECT item_id,rate,settlement_cycle INTO v_item,v_rate,v_cycle FROM policy WHERE
id=p_policy;

 IF v_item IS NULL OR p_orders IS NULL OR JSON_TYPE(p_orders)<>'ARRAY' OR
JSON_LENGTH(p_orders)=0 THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='CLAIM_INPUT_INVALID'; END IF;

 IF v_cycle='MONTH' AND p_period REGEXP '^[0-9]{4}-(0[1-9]|1[0-2])$' THEN

  SET v_start=STR_TO_DATE(CONCAT(p_period,'-01'),'%Y-%m-%d'); SET
v_end=DATE_ADD(v_start,INTERVAL 1 MONTH);

 ELSEIF v_cycle='QUARTER' AND p_period REGEXP '^[0-9]{4}-Q[1-4]$' THEN
```

```
 SET v_start=STR_TO_DATE(CONCAT(LEFT(p_period,4),'-',1+(CAST(RIGHT(p_period,1) AS UNSIGNED)-1)*3,'-01'),'%Y-%m-%d'); SET v_end=DATE_ADD(v_start,INTERVAL 3 MONTH);

 ELSEIF v_cycle='YEAR' AND p_period REGEXP '^[0-9]{4}$' THEN

 SET v_start=STR_TO_DATE(CONCAT(p_period,'-01-01'),'%Y-%m-%d'); SET v_end=DATE_ADD(v_start,INTERVAL 1 YEAR);

 ELSE SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='CLAIM_PERIOD_INVALID'; END IF;

 IF v_start IS NULL OR v_end IS NULL OR EXISTS(SELECT 1 FROM claim WHERE elder_id=p_elder AND item_id=v_item AND period_start<v_end AND period_end>v_start) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='CLAIM_PERIOD_OVERLAP'; END IF;

 INSERT INTO claim(elder_id,item_id,period,community_id,period_start,period_end) VALUES(p_elder,v_item,p_period,v_community,v_start,v_end);

 SET v_claim=LAST_INSERT_ID();

 WHILE v_i<JSON_LENGTH(p_orders) DO

 SET v_order=CAST(JSON_UNQUOTE(JSON_EXTRACT(p_orders,CONCAT('$[',v_i,']'))) AS UNSIGNED);

 IF NOT EXISTS(SELECT 1 FROM service_order o JOIN policy p ON p.id=p_policy JOIN elder e ON e.id=o.elder_id JOIN service_item si ON si.id=o.item_id WHERE o.id=v_order AND o.elder_id=p_elder AND o.item_id=v_item AND o.status='COMPLETED' AND EXISTS(SELECT 1 FROM policy_area pa WHERE pa.policy_id=p.id AND pa.community_id=o.community_id) AND o.starts_at>=p.starts_at AND o.ends_at<=p.ends_at AND o.starts_at>=v_start AND o.starts_at<v_end AND si.subsidizable=1 AND TIMESTAMPDIFF(YEAR,e.birth_date,o.starts_at)>=p.minimum_age AND (p.required_tag IS NULL OR EXISTS(SELECT 1 FROM elder_tag t WHERE t.elder_id=e.id AND t.tag=p.required_tag)) AND EXISTS(SELECT 1 FROM assessment a WHERE a.elder_id=e.id AND a.ability_level=p.ability_level AND a.assessed_at<=o.starts_at AND a.valid_until>=o.ends_at AND NOT EXISTS(SELECT 1 FROM assessment newer WHERE newer.elder_id=e.id AND newer.assessed_at<=o.starts_at AND (newer.assessed_at>a.assessed_at OR (newer.assessed_at=a.assessed_at AND
```

```
newer.id>a.id))))) THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='CLAIM_ORDER_INELIGIBLE'; END IF;

 SET v_minutes=(SELECT a.effective_minutes FROM attendance a JOIN
v_dispatch_context d ON d.id=a.dispatch_id JOIN service_record r ON r.dispatch_id=d.id
WHERE d.order_id=v_order AND d.status='FINISHED' AND a.status='VALID' AND
r.confirmed_by IS NOT NULL LIMIT 1);

 IF v_minutes IS NULL OR v_minutes<=0 THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='CLAIM_EVIDENCE_MISSING'; END IF;

 SELECT o.amount,si.unit INTO v_price,v_unit FROM service_order o JOIN service_item si
ON si.id=o.item_id WHERE o.id=v_order;

 SET v_amount=LEAST(v_price,ROUND(IF(v_unit='HOUR',v_minutes/60*v_rate,v_rate),2));

 IF v_amount<=0 THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='CLAIM_ZERO_AMOUNT'; END IF;

 INSERT INTO
claim_item(claim_id,policy_id,order_id,amount,effective_minutes,rate_snapshot)
VALUES(v_claim,p_policy,v_order,v_amount,v_minutes,v_rate);

 SET v_i=v_i+1;

 END WHILE;

 UPDATE claim SET amount=(SELECT SUM(amount) FROM claim_item WHERE
claim_id=v_claim),status='SUBMITTED' WHERE id=v_claim;

END$$

CREATE TRIGGER claim_before_update BEFORE UPDATE ON claim FOR EACH ROW

BEGIN

 IF OLD.status='PAID' THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='PAID_CLAIM_IMMUTABLE'; END IF;

 IF NEW.status<>OLD.status AND NOT ((OLD.status='DRAFT' AND
NEW.status='SUBMITTED') OR (OLD.status='SUBMITTED' AND NEW.status
IN('STREET_APPROVED','REJECTED')) OR (OLD.status='STREET_APPROVED' AND
NEW.status IN('APPROVED','REJECTED')) OR (OLD.status='APPROVED' AND
NEW.status='PAID') OR (OLD.status='REJECTED' AND NEW.status='SUBMITTED' AND
```

```
EXISTS(SELECT 1 FROM claim_evidence ce WHERE ce.claim_id=OLD.id AND
ce.created_at>=COALESCE((SELECT MAX(created_at) FROM claim_review WHERE
claim_id=OLD.id),'1000-01-01')))) THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='CLAIM_STATE_INVALID'; END IF;

 IF NEW.status='PAID' AND NOT EXISTS(SELECT 1 FROM payment WHERE
claim_id=NEW.id AND amount=NEW.amount) THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='PAYMENT_EVIDENCE_REQUIRED'; END IF;

 IF NEW.status<>'DRAFT' AND NEW.amount<>(SELECT COALESCE(SUM(amount),0)
FROM claim_item WHERE claim_id=NEW.id) THEN SIGNAL SQLSTATE '45000' SET
MESSAGE_TEXT='CLAIM_AMOUNT_MISMATCH'; END IF;

END$$

DELIMITER ;

DELIMITER $$
```

测试与结果

真实 MySQL 集成测试共 125 项，失败 0 项、错误 0 项、跳过 0 项。

WorkflowIntegrationTest：43 项。

HistoryRulesIntegrationTest：16 项。

DatabaseIntegrationTest：66 项。

测试连接独立的真实 MySQL 测试库。覆盖全部表最低记录数、15 组查询等价、视图执行、外键与唯一性、有效期、重叠区间、派单容量、确认凭据、工时复核、设备历史、异常事件、补贴回滚、身份隔离、登录锁定、会话退出及并发。补充测试覆盖招聘录用和拒绝、培训成绩边界、投诉完整办理、比例抽查、整改退回重提、机构月报、公示隐私、跨社区权限、定时模拟、坐标校验、三种补贴周期和补件重报。完整方法名和原始结果保存在 Surefire XML 与测试源代码中。

验证对象	实际验证内容
并发计划	两个事务同时创建同老人重叠计划，只成功一个
并发派单	两个订单竞争最后一个班次名额，只成功一个
失败改派	新班次不覆盖预约时刻时，旧派单和名额被回滚恢复
批量写入	一条合法、一条越权的数据批次整体回滚
完整服务	接单、签到、异常工时复核、签退、家属确认走通
历史保护	终态派单和应急升级日志不能改写或删除
复评与迁移	旧计划暂停后按最新评估修订；设备先解绑再迁移
备份恢复	全量加实际 binlog 增量在独立库恢复，47 表逐行内容一致

浏览器验证覆盖六种角色的 109 个角色查询入口，以及 26 次实际表单操作。投诉、培训、考核、整改、公示和模拟采样均完成提交；详情、免登录公示和手机宽度布局一并检查。



图 29 浏览器中的投诉办理记录

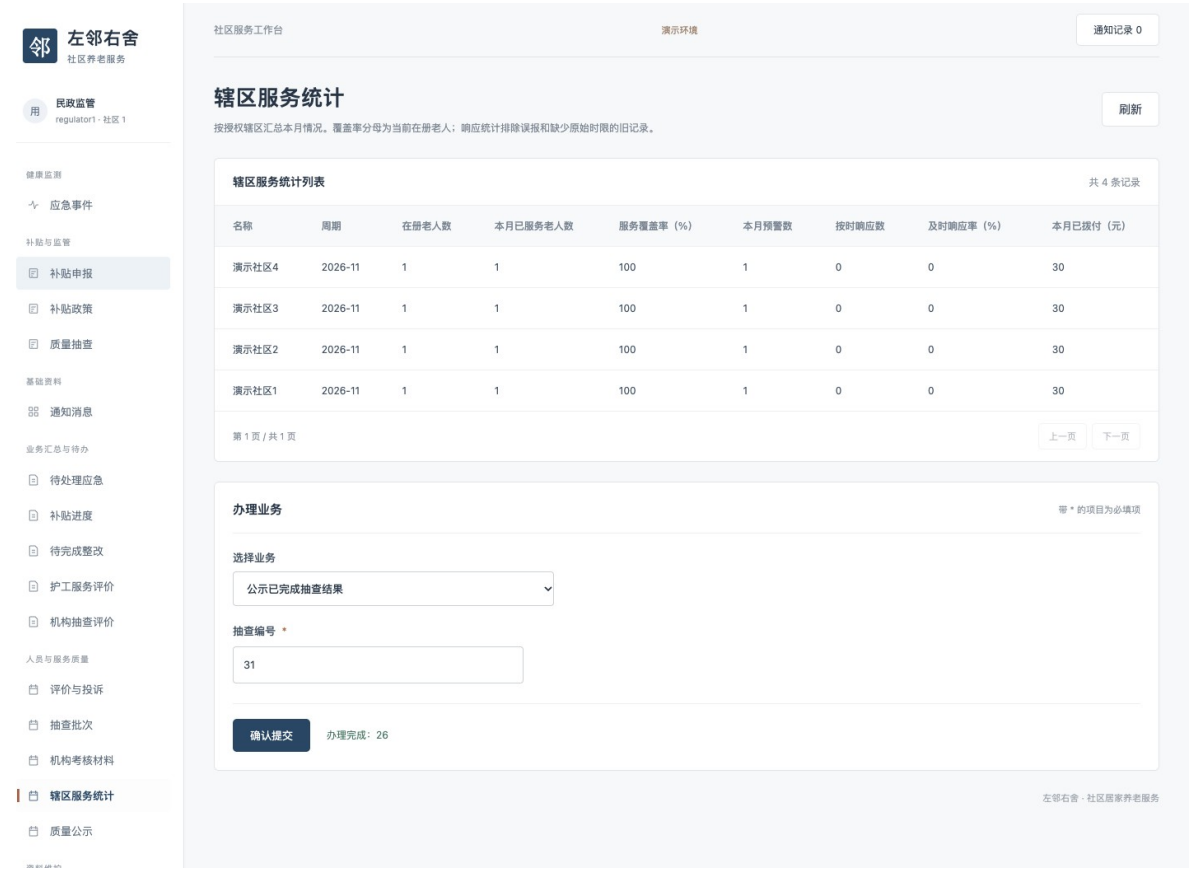


图 30 民政监管按授权辖区查看统计

七、附件内容

附件编号	内容与位置
01	docs/diagrams：全局与局部 UML、用例图、鲁棒图，含 DOT、SVG、PNG
02	database：全部 DDL、视图、过程、触发器、15 组双实现查询
03	src 与 pom.xml：Java 源码、静态界面和测试源码
04	docs/数据字典与关系模式.md：所有字段的名称、类型、默认值、关联与含义
05	evidence：测试日志、查询执行计划、行数、恢复演练、浏览器截图
06	eldercare.jar 与 Windows 初始化、启动脚本
07	答辩演示文稿与 5 分钟录制讲稿；【待补充】全员发言视频
08	个人报告预填稿；【待确认】个人分工和完成情况
09	【待补充】答辩后的教师提问、修改内容及验证结果

【待补充】成员与分工确认后更新个人报告，补入协作记录、答辩录像和教师反馈。

完整字段字典

以下字段依据实际 MySQL 结构导出。类型中的枚举值给出允许的状态范围，空值、默认值、主外键及检查条件分别列出；含义不明的字段不以英文名代替说明。

社区 community

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
name	varchar(80)	无默认值	非空	业务对象名称
district	varchar(80)	无默认值	非空	所属演示区县
area_code	varchar(30)	无默认值	唯一；非空	社区服务范围编码
street_name	varchar(80)	演示街道	非空	所属街道名称
center_x	decimal(12,3)	0.000	非空	虚构平面坐标中心横坐标，米
center_y	decimal(12,3)	0.000	非空	虚构平面坐标中心纵坐标，米
radius_m	int	5000	非空	社区服务范围半径，必须为正数
fence_m	int	200	非空	签到签退允许距离，必须为正数

检查条件：((`radius_m` > 0) and (`fence_m` > 0) and (`fence_m` <= `radius_m`))

服务机构 organization

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
name	varchar(100)	无默认值	非空	业务对象名称
community_id	bigint	无默认值	非空；FK community.id	社区或授权辖区编号

账号 account

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
username	varchar(50)	无默认值	唯一；非空	登录账号名
password_hash	varchar(100)	无默认值	非空	BCrypt 加盐密码摘要
role	enum('ADMIN','FAMILY','CAREGIVER','MANAGER','STREET','REGULATOR')	无默认值	非空	业务角色类别
community_id	bigint	无默认值	非空；FK community.id	社区或授权辖区编号
organization_id	bigint	NULL	可空；FK organization.id	服务机构或机构授权范围编号
failures	int	0	非空	累计连续登录失败次数

字段	类型及范围	默认值	约束	含义
locked_until	datetime	NULL	可空	登录限制截止时间
active	tinyint(1)	1	非空	账号或家属关联是否有效

老人档案 elder

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
name	varchar(60)	无默认值	非空	业务对象名称
identity_hash	char(64)	无默认值	唯一；非空	身份证 HMAC 查重值
identity_cipher	text	无默认值	非空	AES GCM 加密身份信息
birth_date	date	无默认值	非空	出生日期
address	varchar(200)	无默认值	非空	档案地址或订单发生时的地址快照
community_id	bigint	无默认值	非空；FK community.id	社区或授权辖区编号
status	enum('ACTIVE','INACTIVE')	ACTIVE	非空	业务状态，允许值见枚举
position_x	decimal(12,3)	0.000	非空	档案或订单住址横坐标，米
position_y	decimal(12,3)	0.000	非空	档案或订单住址纵坐标，米

老人标签 elder_tag

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
elder_id	bigint	无默认值	非空；FK elder.id	老人档案编号
tag	enum('ALONE','DISABLED','LOW_INCOME')	无默认值	非空	老人特殊标签

家属关联 guardian

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
elder_id	bigint	无默认值	非空；FK elder.id	老人档案编号
account_id	bigint	无默认值	非空；FK account.id	关联账号编号
relation_name	varchar(30)	无默认值	非空	家属与老人关系
phone_cipher	text	无默认值	非空	加密联系电话
active	tinyint(1)	1	非空	账号或家属关联是否有效

能力评估 assessment

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
elder_id	bigint	无默认值	非空；FK elder.id	老人档案编号
scale_name	varchar(50)	无默认值	非空	能力评估量表名称
score	int	无默认值	非空	评估或抽查得分
ability_level	enum('SELF','PARTIAL','SEVERE')	无默认值	非空	能力等级
assessor_id	bigint	无默认值	非空；FK account.id	评估人员账号
assessed_at	datetime	无默认值	非空	评估发生时间
valid_until	datetime	无默认值	非空	有效期结束时刻

检查条件：(`score` between 0 and 100)

检查条件：(`valid_until` > `assessed_at`)

服务项目 service_item

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号

字段	类型及范围	默认值	约束	含义
name	varchar(80)	无默认值	非空	业务对象名称
category	varchar(30)	无默认值	非空	服务类别
unit	enum('VISIT','HOUR')	无默认值	非空	计费单位，按次或小时
price	decimal(10,2)	无默认值	非空	服务目录指导价
duration_minutes	int	无默认值	非空	标准服务分钟数
subsidizable	tinyint(1)	无默认值	非空	是否允许补贴

检查条件：(`price` >= 0)

检查条件：(`duration_minutes` > 0)

照护服务包 care_bundle

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
name	varchar(80)	无默认值	非空	业务对象名称
ability_level	enum('SELF','PARTIAL','SEVERE')	无默认值	非空	能力等级

服务包明细 bundle_item

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
bundle_id	bigint	无默认值	非空；FK care_bundle.id	照护服务包编号
item_id	bigint	无默认值	非空；FK service_item.id	服务项目编号
weekly_frequency	int	无默认值	非空	每周服务频次

检查条件：(`weekly_frequency` > 0)

护工 caregiver

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
account_id	bigint	无默认值	唯一；非空；FK account.id	关联账号编号
organization_id	bigint	无默认值	非空；FK organization.id	服务机构或机构授权范围编号
name	varchar(60)	无默认值	非空	业务对象名称
status	enum('ACTIVE','LEAVE','LEFT')	ACTIVE	非空	业务状态，允许值见枚举

项目资质 credential

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
caregiver_id	bigint	无默认值	非空；FK caregiver.id	护工编号
item_id	bigint	无默认值	非空；FK service_item.id	服务项目编号
certificate_no	varchar(80)	无默认值	唯一；非空	职业证书唯一编号
certificate_level	varchar(30)	无默认值	非空	证书等级说明
valid_from	datetime	无默认值	非空	资质开始生效时刻
valid_until	datetime	无默认值	非空	有效期结束时刻

检查条件：(`valid_until` > `valid_from`)

护工服务区域 caregiver_area

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
caregiver_id	bigint	无默认值	非空；FK caregiver.id	护工编号
community_id	bigint	无默认值	非空；FK community.id	社区或授权辖区编号

照护计划 care_plan

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
assessment_id	bigint	无默认值	非空；FK assessment.id	计划采用的评估记录编号
bundle_id	bigint	无默认值	非空；FK care_bundle.id	照护服务包编号
caregiver_id	bigint	无默认值	非空；FK caregiver.id	护工编号
starts_at	datetime	无默认值	非空	区间开始时刻，包含边界
ends_at	datetime	无默认值	非空	区间结束时刻，不含边界
status	enum('ACTIVE','PAUSED','EXPIRED','ENDED')	ACTIVE	非空	业务状态，允许值见枚举

检查条件：(`ends_at` > `starts_at`)

计划明细 plan_item

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
plan_id	bigint	无默认值	非空；FK care_plan.id	照护计划编号
item_id	bigint	无默认值	非空；FK service_item.id	服务项目编号

字段	类型及范围	默认值	约束	含义
weekly_frequency	int	无默认值	非空	每周服务频次

检查条件：(`weekly_frequency` > 0)

排班 shift

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
caregiver_id	bigint	无默认值	非空；FK caregiver.id	护工编号
starts_at	datetime	无默认值	非空	区间开始时刻，包含边界
ends_at	datetime	无默认值	非空	区间结束时刻，不含边界
capacity	int	无默认值	非空	班次可接单数量
used_count	int	0	非空	已占用名额缓存
status	enum('OPEN','CLOSED')	OPEN	非空	业务状态，允许值见枚举

检查条件：(`ends_at` > `starts_at`)

检查条件：((`capacity` > 0) and (`used_count` >= 0) and (`used_count` <= `capacity`))

服务订单 service_order

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号

字段	类型及范围	默认值	约束	含义
elder_id	bigint	无默认值	非空；FK elder.id	老人档案编号
item_id	bigint	无默认值	非空；FK service_item.id	服务项目编号
community_id	bigint	无默认值	非空；FK community.id	社区或授权辖区编号
starts_at	datetime	无默认值	非空	区间开始时刻，包含边界
ends_at	datetime	无默认值	非空	区间结束时刻，不含边界
address	varchar(200)	无默认值	非空	档案地址或订单发生时的地址快照
channel	enum('PHONE','FAMILY','COMMUNITY')	无默认值	非空	下单来源
amount	decimal(10,2)	无默认值	非空	订单价格、申报金额或拨付金额
status	enum('PENDING','ASSIGNED','ACCEPTED','SERVING','CONFIRM','COMPLETED','CANCELLED')	PENDING	非空	业务状态，允许值见枚举
created_by	bigint	无默认值	非空；FK account.id	创建人账号
created_at	datetime	当前时间	非空	记录创建时间

字段	类型及范围	默认值	约束	含义
position_x	decimal(12,3)	0.000	非空	档案或订单住址横坐标，米
position_y	decimal(12,3)	0.000	非空	档案或订单住址纵坐标，米

检查条件：(`ends_at` > `starts_at`)

检查条件：(`amount` >= 0)

派单记录 dispatch

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
order_id	bigint	无默认值	非空；FK service_order.id	服务订单编号
shift_id	bigint	无默认值	非空；FK shift.id	护工排班编号
status	enum('WAITING','ACCEPTED','FINISHED','REASSIGNED','REJECTED')	WAITING	非空	业务状态，允许值见枚举
reason	varchar(200)	首次派单	非空	改派、审核或业务操作原因
assigned_at	datetime	当前时间	非空	派单发生时间
accepted_at	datetime	NULL	可空	护工接单时间
active_order	bigint	NULL	唯一；可空	仅有效派单有值的生成列，保障一单一有效派单

服务与确认 service_record

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
dispatch_id	bigint	无默认值	唯一；非空；FK dispatch.id	本次派单编号
notes	varchar(1000)	无默认值	非空	服务内容与过程说明
confirmed_by	bigint	NULL	可空；FK account.id	老人或家属确认账号
confirmed_at	datetime	NULL	可空	服务确认时间
confirmation	varchar(300)	NULL	可空	确认凭据说明

工时记录 attendance

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
dispatch_id	bigint	无默认值	唯一；非空；FK dispatch.id	本次派单编号
check_in	datetime(6)	无默认值	非空	签到时间
check_out	datetime(6)	NULL	可空	签退时间
location_note	varchar(200)	无默认值	非空	位置说明或模拟定位结果

字段	类型及范围	默认值	约束	含义
fence_ok	tinyint(1)	无默认值	非空	服务端计算的签到围栏结果；旧记录保留原值
status	enum('PENDING','REVIEW','VALID','INVALID')	PENDING	非空	业务状态，允许值见枚举
effective_minutes	int	0	非空	核定有效工时，单位分钟
reviewer_id	bigint	NULL	可空；FK account.id	复核或审核人员账号
review_reason	varchar(300)	NULL	可空	工时复核原因
checkin_x	decimal(12,3)	NULL	可空	签到横坐标，米
checkin_y	decimal(12,3)	NULL	可空	签到纵坐标，米
checkout_x	decimal(12,3)	NULL	可空	签退横坐标，米
checkout_y	decimal(12,3)	NULL	可空	签退纵坐标，米
checkout_fence_ok	tinyint(1)	NULL	可空	服务端计算的签退围栏结果

检查条件：((`check_out` is null) or (`check_out` > `check_in`))

检查条件：(`effective_minutes` >= 0)

检查条件：((`checkin_x` is null) = (`checkin_y` is null))

检查条件：((`checkout_x` is null) = (`checkout_y` is null))

设备 device

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号

字段	类型及范围	默认值	约束	含义
serial_no	varchar(80)	无默认值	唯一；非空	设备唯一序列号
type_name	varchar(50)	无默认值	非空	设备类型
location_note	varchar(200)	无默认值	非空	位置说明或模拟定位结果
status	enum('ACTIVE','OFFLINE','RETIRED')	ACTIVE	非空	业务状态，允许值见枚举
community_id	bigint	NULL	可空；FK community.id	社区或授权辖区编号
simulation_enabled	tinyint(1)	0	非空	是否允许定时模拟采样，默认关闭
simulation_metric_id	bigint	NULL	可空；FK metric_rule.id	该设备模拟采样采用的指标规则
simulation_seconds	int	60	非空	模拟采样间隔，30 至 3600 秒
last_simulated_at	datetime(6)	NULL	可空	上一次定时模拟采样时刻

检查条件：(`simulation_seconds` between 30 and 3600)

绑定历史 device_binding

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
device_id	bigint	无默认值	非空；FK device.id	监测设备编号

字段	类型及范围	默认值	约束	含义
elder_id	bigint	无默认值	非空；FK elder.id	老人档案编号
bound_at	datetime(6)	无默认值	非空	绑定开始时间
unbound_at	datetime(6)	NULL	可空	解绑时间，为空表示当前绑定
active_device	bigint	NULL	唯一；可空	当前绑定设备的生成唯一列

检查条件：((`unbound_at` is null) or (`unbound_at` > `bound_at`))

指标规则 metric_rule

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
metric	varchar(60)	无默认值	唯一；非空	课程模拟指标名称
low_value	decimal(12,3)	无默认值	非空	演示阈值下界
high_value	decimal(12,3)	无默认值	非空	演示阈值上界
response_seconds	int	无默认值	非空	初始应急响应时限秒数

检查条件：(`high_value` > `low_value`)

检查条件：(`response_seconds` > 0)

体征数据 vital

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
binding_id	bigint	无默认值	非空；FK device_binding.id	采集时有效的绑定历史编号
metric_rule_id	bigint	无默认值	非空；FK metric_rule.id	所用指标规则编号
sample_key	varchar(80)	无默认值	非空	设备侧采样幂等标识
collected_at	datetime(6)	无默认值	非空	实际采集时间
metric_value	decimal(12,3)	无默认值	非空	采集指标数值
abnormal	tinyint(1)	0	非空	是否越过配置阈值
source	varchar(50)	无默认值	非空	采集或应急触发来源

应急事件 emergency

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
elder_id	bigint	无默认值	非空；FK elder.id	老人档案编号
vital_id	bigint	NULL	唯一；可空；FK vital.id	触发事件的体征记录

字段	类型及范围	默认值	约束	含义
source	varchar(50)	无默认值	非空	采集或应急触发来源
level	int	1	非空	应急级别或升级级别
occurred_at	datetime	当前时间	非空	事件发生时间
deadline	datetime	无默认值	非空	本级响应截止时间
status	enum('UNCONFIRMED','OPEN','RESPONDED','CLOSED','FALSE_ALARM')	无默认值	非空	业务状态，允许值见枚举
responded_by	bigint	NULL	可空；FK account.id	响应人员账号
responded_at	datetime	NULL	可空	首次响应时间
arrived_at	datetime	NULL	可空	到场时间
result	varchar(1000)	NULL	可空	处置或整改复核结果
initial_deadline	datetime	NULL	可空	首次响应截止时间；旧库未知值保留空
closed_at	datetime(6)	NULL	可空	事件关闭时间，与到场时间分别保存
close_reason	varchar(300)	NULL	可空	误报关闭的核实说明

检查条件：(`level` between 1 and 3)

升级记录 emergency_log

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
event_id	bigint	无默认值	非空；FK emergency.id	应急事件编号
level	int	无默认值	非空	应急级别或升级级别
created_at	datetime	当前时间	非空	记录创建时间
message	varchar(300)	无默认值	非空	通知或事件日志内容

通知 notification

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
event_id	bigint	NULL	可空；FK emergency.id	应急事件编号
account_id	bigint	无默认值	非空；FK account.id	关联账号编号
message	varchar(300)	无默认值	非空	通知或事件日志内容
created_at	datetime	当前时间	非空	记录创建时间

字段	类型及范围	默认值	约束	含义
assessment_id	bigint	NULL	可空；FK assessment.id	计划采用的评估记录编号
dedupe_key	varchar(150)	NULL	唯一；可空	复评通知去重标识

补贴政策 policy

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
name	varchar(100)	无默认值	非空	业务对象名称
community_id	bigint	无默认值	非空；FK community.id	社区或授权辖区编号
item_id	bigint	无默认值	非空；FK service_item.id	服务项目编号
ability_level	enum('SELF','PARTIAL','SEVERE')	无默认值	非空	能力等级
rate	decimal(10,2)	无默认值	非空	模拟补贴单价
starts_at	datetime	无默认值	非空	区间开始时刻，包含边界
ends_at	datetime	无默认值	非空	区间结束时刻，不含边界
settlement_cycle	enum('MONTH','QUARTER','YEAR')	MONTH	非空	结算周期，月 / 季 / 年
minimum_age	int	0	非空	政策最低年龄，0 至 120

字段	类型及范围	默认值	约束	含义
required_tag	enum('ALONE','DISABLED','LOW_INCOME')	NULL	可空	政策要求的老人标签，为空不限制

检查条件：(`rate` >= 0)

检查条件：(`ends_at` > `starts_at`)

检查条件：(`minimum_age` between 0 and 120)

补贴申报 claim

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
elder_id	bigint	无默认值	非空；FK elder.id	老人档案编号
item_id	bigint	无默认值	非空；FK service_item.id	服务项目编号
period	varchar(10)	无默认值	非空	申报按政策采用 YYYY-MM、YYYY-Qn 或 YYYY；机构月报采用 YYYY-MM
amount	decimal(12,2)	0.00	非空	订单价格、申报金额或拨付金额
status	enum('DRAFT','SUBMITTED','STREET_APPROVED','APPROVED','REJECTED','PAID')	DRAFT	非空	业务状态，允许值见枚举

字段	类型及范围	默认值	约束	含义
community_id	bigint	NULL	可空；FK community.id	社区或授权辖区编号
period_start	date	NULL	可空	申报周期开始日期，包含
period_end	date	NULL	可空	申报周期截止日期，不含

检查条件：(`amount` >= 0)

检查条件：((`period_end` is null) or (`period_end` > `period_start`))

申报明细 claim_item

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
claim_id	bigint	无默认值	非空；FK claim.id	申报编号
policy_id	bigint	无默认值	非空；FK policy.id	适用政策编号
order_id	bigint	无默认值	唯一；非空；FK service_order.id	服务订单编号
amount	decimal(12,2)	无默认值	非空	订单价格、申报金额或拨付金额
effective_minutes	int	无默认值	非空	核定有效工时，单位分钟
rate_snapshot	decimal(10,2)	无默认值	非空	核算时采用的单价快照

检查条件：((`amount` >= 0) and (`effective_minutes` > 0))

审核记录 claim_review

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
claim_id	bigint	无默认值	非空；FK claim.id	申报编号
reviewer_id	bigint	无默认值	非空；FK account.id	复核或审核人员账号
stage	enum('STREET','REGULATOR')	无默认值	非空	街道或民政审核阶段
decision	enum('APPROVE','REJECT')	无默认值	非空	通过或驳回
reason	varchar(300)	无默认值	非空	改派、审核或业务操作原因
created_at	datetime	当前时间	非空	记录创建时间

拨付 payment

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
claim_id	bigint	无默认值	唯一；非空；FK claim.id	申报编号
reference_no	varchar(80)	无默认值	唯一；非空	模拟拨付凭证唯一编号

字段	类型及范围	默认值	约束	含义
amount	decimal(12,2)	无默认值	非空	订单价格、申报金额或拨付金额
paid_at	datetime	当前时间	非空	模拟拨付时间

检查条件：(`amount` > 0)

质量抽查 inspection

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
order_id	bigint	NULL	可空；FK service_order.id	服务订单编号
caregiver_id	bigint	NULL	可空；FK caregiver.id	护工编号
inspector_id	bigint	无默认值	非空；FK account.id	抽查人员账号
method	enum('PHONE','VISIT')	无默认值	非空	电话回访或上门核查
score	int	NULL	可空	评估或抽查得分
issue	varchar(300)	无默认值	非空	问题说明
rectification	varchar(300)	无默认值	非空	整改要求
status	enum('PENDING','OPEN','REVIEW','CLOSED')	OPEN	非空	业务状态，允许值见枚举

字段	类型及范围	默认值	约束	含义
result	varchar(300)	NULL	可空	处置或整改复核结果
batch_id	bigint	NULL	可空；FK inspection_batch.id	所属抽查批次
created_at	datetime	当前时间	非空	记录创建时间

检查条件：((`order_id` is null) <> (`caregiver_id` is null))

检查条件：(`score` between 0 and 100)

评价投诉 complaint

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
order_id	bigint	无默认值	非空；FK service_order.id	服务订单编号
account_id	bigint	无默认值	非空；FK account.id	关联账号编号
rating	int	无默认值	非空	家属评价分数，1 至 5
content	varchar(500)	无默认值	非空	评价或投诉内容
created_at	datetime	当前时间	非空	记录创建时间
status	enum('SUBMITTED','ASSIGNED','REPLIED','CLOSED')	SUBMITTED	非空	业务状态，允许值见枚举

字段	类型及范围	默认值	约束	含义
organization_id	bigint	NULL	可空；FK organization.id	服务机构或机构授权范围编号
handler_id	bigint	NULL	可空；FK account.id	投诉受理经办账号
result	varchar(1000)	NULL	可空	处置或整改复核结果
updated_at	datetime	当前时间	非空	最近一次状态变更时间
kind	enum('REVIEW','COMPLAINT')	COMPLAINT	非空	普通评价或待办理投诉

检查条件：(`rating` between 1 and 5)

操作审计 audit_log

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
table_name	varchar(50)	无默认值	非空	操作对象表名
row_id	bigint	无默认值	非空	操作对象记录号
operation	varchar(20)	无默认值	非空	新增或修改动作
actor	varchar(100)	无默认值	非空	业务操作者或数据库会话来源
old_status	varchar(100)	NULL	可空	变更前状态
new_status	varchar(100)	NULL	可空	变更后状态

字段	类型及范围	默认值	约束	含义
created_at	datetime	当前时间	非空	记录创建时间
before_data	json	NULL	可空	修改前非秘密字段快照
after_data	json	NULL	可空	修改后字段或操作原因快照

监管辖区授权 account_scope

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
account_id	bigint	无默认值	非空；FK account.id	关联账号编号
community_id	bigint	无默认值	非空；FK community.id	社区或授权辖区编号

政策适用社区 policy_area

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
policy_id	bigint	无默认值	非空；FK policy.id	适用政策编号
community_id	bigint	无默认值	非空；FK community.id	社区或授权辖区编号

招聘申请 recruitment

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
organization_id	bigint	无默认值	非空；FK organization.id	服务机构或机构授权范围编号
candidate_name	varchar(60)	无默认值	非空	应聘人姓名
phone_cipher	text	无默认值	非空	加密联系电话
status	enum('APPLIED','HIRED','REJECTED')	APPLIED	非空	业务状态，允许值见枚举
caregiver_id	bigint	NULL	唯一；可空；FK caregiver.id	护工编号
reviewer_id	bigint	NULL	可空；FK account.id	复核或审核人员账号
review_note	varchar(500)	NULL	可空	招聘、整改或考核审核意见
created_at	datetime	当前时间	非空	记录创建时间
reviewed_at	datetime	NULL	可空	审核完成时间

岗前培训 training

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号

字段	类型及范围	默认值	约束	含义
organization_id	bigint	无默认值	非空；FK organization.id	服务机构或机构授权范围编号
title	varchar(100)	无默认值	非空	培训或公示标题
starts_at	datetime	无默认值	非空	区间开始时刻，包含边界
ends_at	datetime	无默认值	非空	区间结束时刻，不含边界

检查条件：(`ends_at` > `starts_at`)

培训成绩 training_record

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
training_id	bigint	无默认值	非空；FK training.id	培训安排编号
caregiver_id	bigint	无默认值	非空；FK caregiver.id	护工编号
result	enum('PENDING','PASS','FAIL')	PENDING	非空	处置或整改复核结果
score	int	NULL	可空	评估或抽查得分
recorded_by	bigint	NULL	可空；FK account.id	成绩登记人账号
recorded_at	datetime	NULL	可空	成绩登记时间

检查条件：((`score` is null) or (`score` between 0 and 100))

投诉办理记录 complaint_log

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
complaint_id	bigint	无默认值	非空；FK complaint.id	评价或投诉编号
actor_id	bigint	无默认值	非空；FK account.id	办理人员账号
status	varchar(30)	无默认值	非空	业务状态，允许值见枚举
message	varchar(1000)	无默认值	非空	通知或事件日志内容
created_at	datetime	当前时间	非空	记录创建时间

比例抽查批次 inspection_batch

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
community_id	bigint	无默认值	非空；FK community.id	社区或授权辖区编号
inspector_id	bigint	无默认值	非空；FK account.id	抽查人员账号
target_type	enum('ORDER','CAREGIVER')	无默认值	非空	抽查对象类别，订单或护工

字段	类型及范围	默认值	约束	含义
starts_at	datetime	无默认值	非空	区间开始时刻，包含边界
ends_at	datetime	无默认值	非空	区间结束时刻，不含边界
ratio	decimal(6,5)	无默认值	非空	抽样比例，大于 0 且不超过 1
population	int	无默认值	非空	抽样时符合条件的对象数
sample_count	int	无默认值	非空	按比例向上取整的实际样本数
sample_seed	char(36)	无默认值	非空	选择样本时使用的随机种子
created_at	datetime	当前时间	非空	记录创建时间

检查条件：(`ends_at` > `starts_at`)

检查条件：((`ratio` > 0) and (`ratio` <= 1))

检查条件：((`population` >= `sample_count`) and (`sample_count` > 0))

整改提交与复核 rectification_submission

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
inspection_id	bigint	无默认值	非空；FK inspection.id	抽查编号

字段	类型及范围	默认值	约束	含义
submitted_by	bigint	无默认值	非空；FK account.id	提交材料的账号
measures	varchar(1000)	无默认值	非空	机构提出并执行的整改措施
evidence	varchar(1000)	无默认值	非空	证明材料内容或业务记录说明
created_at	datetime	当前时间	非空	记录创建时间
decision	enum('PENDING','ACCEPT','RETURN')	PENDING	非空	通过或驳回
reviewer_id	bigint	NULL	可空；FK account.id	复核或审核人员账号
review_note	varchar(1000)	NULL	可空	招聘、整改或考核审核意见
reviewed_at	datetime	NULL	可空	审核完成时间

机构考核材料 organization_report

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
organization_id	bigint	无默认值	非空；FK organization.id	服务机构或机构授权范围编号
period	char(7)	无默认值	非空	申报按政策采用 YYYY-MM、YYYY-Qn 或 YYYY；机构月报采用 YYYY-MM

字段	类型及范围	默认值	约束	含义
summary	varchar(2000)	无默认值	非空	机构月度情况说明
evidence	varchar(1000)	无默认值	非空	证明材料内容或业务记录说明
submitted_by	bigint	无默认值	非空；FK account.id	提交材料的账号
status	enum('SUBMITTED','ACCEPTED','RETURNED')	SUBMITTED	非空	业务状态，允许值见枚举
reviewer_id	bigint	NULL	可空；FK account.id	复核或审核人员账号
review_note	varchar(1000)	NULL	可空	招聘、整改或考核审核意见
created_at	datetime	当前时间	非空	记录创建时间
reviewed_at	datetime	NULL	可空	审核完成时间

服务质量公示 public_notice

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
inspection_id	bigint	无默认值	非空；FK inspection.id	抽查编号
publisher_id	bigint	无默认值	非空；FK account.id	公示发布人账号

字段	类型及范围	默认值	约束	含义
title	varchar(100)	无默认值	非空	培训或公示标题
body	varchar(1000)	无默认值	非空	不包含身份、住址和健康明细的公示文本
published_at	datetime	当前时间	非空	公示发布时间
withdrawn_at	datetime	NULL	可空	撤回时间，空表示仍在公示
withdraw_reason	varchar(300)	NULL	可空	撤回公示的原因

补贴证明材料 claim_evidence

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
claim_id	bigint	无默认值	非空；FK claim.id	申报编号
submitted_by	bigint	无默认值	非空；FK account.id	提交材料的账号
name	varchar(100)	无默认值	非空	业务对象名称
content	text	无默认值	非空	评价或投诉内容
content_sha256	char(64)	无默认值	非空	归档材料内容的 SHA-256 摘要

字段	类型及范围	默认值	约束	含义
created_at	datetime	当前时间	非空	记录创建时间

应急处置步骤 event_step

字段	类型及范围	默认值	约束	含义
id	bigint	自动编号	PK；非空	内部唯一编号
event_id	bigint	无默认值	非空；FK emergency.id	应急事件编号
actor_id	bigint	无默认值	非空；FK account.id	办理人员账号
action	varchar(30)	无默认值	非空	应急步骤类别
message	varchar(1000)	无默认值	非空	通知或事件日志内容
created_at	datetime(6)	当前时间	非空	记录创建时间

参考资料

课程设计任务书 2026，第 1—9 页。

教师任务讲解逐字稿，00:01:58—00:34:23。

数据库系统课程设计报告模板 2026 小组与个人版。

数据库系统课程设计评分标准 2026 版。

MySQL 官方文档：CREATE TRIGGER、CREATE PROCEDURE、

SIGNAL，https://dev.mysql.com/doc/refman/8.0/en/create-trigger.html 。

Spring 官方 JDBC 文档：https://docs.spring.io/spring-framework/reference/data-access/jdbc/core.html 。